

IntentChecker: An Intent-Model-Based Debugging Assistant for Mitigating Numeric Logic Errors in Solidity

Inseong Jeon¹, Sundeuk Kim¹, Hyunwoo Kim¹, Hoh Peter In^{1*}

^{1*}Department of Computer Science, Korea University, 145, Anam-ro, Seonbuk-gu, 02841, Seoul, Republic of Korea.

*Corresponding author(s). E-mail(s): hoh_in@korea.ac.kr;
Contributing authors: iwwyou@korea.ac.kr; sd_kim@korea.ac.kr;
khw0809@korea.ac.kr;

Abstract

Smart contract development lacks the mature debugging infrastructure available in traditional software engineering. In particular, developer intent has no direct role in the analysis available at development time. For this reason, numeric logic errors—where program behavior diverges from developer intent—are particularly difficult to address. Existing detection tools instead match completed source code against pre-defined vulnerability patterns in a testing or auditing context. As a result, errors outside these patterns have no practical path to early prevention. This paper presents IntentChecker, a development-time value-level intent validation tool for Solidity that operates at multi-contract single-transaction scope and shifts the detection target from pre-defined patterns to developer-defined intent. At its core is an intent model comprising annotations and a validation engine. Two annotation types let developers declare intent: @During for intended relationships at specific program points within a function, and @Post for intended relationships after function execution. The validation engine classifies each annotation as Satisfied, Warning, or Violated against the intervals computed by abstract interpretation, and reports each result with a risk score. IntentChecker is evaluated on real-world numeric logic errors, where it surfaces a subset of them as Violated judgments at a mean 3.69 s per case, providing feedback complementary to existing auditing-time detectors.

Keywords: Solidity, Numeric Logic Error, Debugging Assistant, Value-level Intent Validation, Intent Model

1 Introduction

The debugging toolchain available to smart contract developers remains far less mature than what traditional software engineering provides. Our prior work, SolQDebug (Jeon et al., 2026), addressed this immaturity by performing statement-level interval-domain abstract interpretation at development time. SolQDebug, however, goes no further than reporting raw intervals—leaving correctness judgment entirely to the developer—and its analysis remains confined to a single contract and a single transaction. This burden is compounded by the widespread use of large language models (LLMs) such as Claude (Anthropic, 2026), ChatGPT (OpenAI, 2026), and Gemini (Google, 2026) for code generation, where the generated code is not verified against its intended behavior.

With correctness judgment left to the developer, logic errors—bugs where program behavior diverges from developer intent—are particularly hard to catch, since neither the compiler nor the runtime raises any signal. A recent empirical study (Chaliasos et al., 2024) confirms this difficulty: practitioners identified logic errors as the most difficult vulnerability type to detect. In response, several tools (Chen et al., 2025; Sun et al., 2024; Zhang, 2024) explicitly target this class of bugs. Yet they all take the source code alone as their input, operating in an auditing or testing context. Moreover, what counts as correct in any given function is set by the developer who wrote it, so by the very definition of a logic error, detecting one requires knowing the developer’s intent. However, current tools take a different approach: they rely on a fixed set of known vulnerabilities chosen by their designers, checking code against a pre-specified set of patterns. This mismatch is most consequential for numeric computation, which sits at the core of most smart contract code: our analysis (IntentChecker, 2026b) of 101 smart contracts from DAppSCAN-source (Zheng et al., 2024), covering 1,023 non-trivial functions, finds numeric types in 82% of them. Logic errors are therefore most likely to arise in this space, and addressing such numeric logic errors calls for a fundamentally different kind of analysis: one that takes developer intent as direct input rather than attempting to recover it from code.

Meanwhile, developer preferences favor tools that run at development time. The same empirical study (Chaliasos et al., 2024) reports a clear preference for IDE-integrated tools such as extensions and linters over standalone analyzers, favoring development-time tooling. It also reports a strong prioritization of low false negatives (92.3%) over low false positives (50%), matching the profile of sound over-approximating analyses such as abstract interpretation. These observations together motivate a development-time tool that runs while the function is being written and lets the developer directly declare the numeric intent their contract encodes, rather than relying on pre-authored pattern catalogs to infer it.

To that end, this paper presents IntentChecker, a development-time value-level intent validation tool for Solidity built on SolQDebug and extended to a multi-contract single-transaction scope. The intent model comprises two annotation types and a validation engine. The two annotation types cover different temporal windows of a function’s execution: @During for relationships that should hold at specific program points during execution, and @Post for relationships that should hold at function exit. The validation engine, grounded in a formal denotational semantics, classifies each annotation as Satisfied, Warning, or Violated against the intervals produced by

abstract interpretation, reporting the result together with a risk score on a 0–10 scale and the sub-intervals where the violation may occur.

IntentChecker is evaluated on 74 effective benchmark instances (89 collected, 15 excluded) drawn from Web3Bugs (Zhang et al., 2023), NumScout (Chen et al., 2025), and the Greedy Contract dataset of Soud et al. (2024). The evaluation is organized around three research questions: whether IntentChecker can surface real-world numeric logic errors as Violated judgments at development-time latency (RQ1), what structural limitations of the analysis engine and the intent model account for the cases it cannot surface (RQ2), and how IntentChecker compares with existing detection tools along the criteria and position axes that define the design space (RQ3). IntentChecker is positioned as a development-time complement to existing auditing-time detectors: it enables intent-driven validation during editing rather than competing with bug-pattern detectors on detection accuracy, and the RQ3 comparison is designed around this complementarity.

Contributions.

- This paper defines numeric logic errors, classifies their representative patterns by abstraction level (value-level vs algorithm-level), and shows that existing detection tools share two structural constraints in addressing this class: detection criteria fixed at design time, and analysis input confined to completed source code at audit time.
- An intent model is presented, consisting of two annotation types and a validation engine grounded in a formal denotational semantics; the engine produces a tri-valued judgment (Satisfied / Warning / Violated) against abstract states, along with a separately computed risk score and violated region.
- IntentChecker is evaluated on 74 effective benchmark instances, of which 46 admit a representable numeric intent and 32 validate as Violated (median 2.11 s per case), and is compared with ScType, GPTScan, and NumScout along the criteria and position axes to substantiate its complementary role as a development-time debugging assistant.
- Building on the evaluation results, this paper turns recurring patterns across the mitigated and L5 cases into three practical guidelines for authoring intent annotations: enumerating expected state effects, anchoring outputs to in-scope quantities, and expressing arithmetic intent at the mathematically ideal level.

The remainder of this paper is organized as follows. Section 2 motivates the development-time intent validation problem and surveys the structural limitations of existing detection tools. Section 3 presents the design of IntentChecker, including the deployment-unit scope, the @During/@Post intent annotations, the @IReturn debug annotation for interface return values, and the validation engine. Section 4 presents evaluation results across the three research questions. Section 5 discusses the intent authoring effort, annotation guidelines, and remaining limitations. Section 6 reviews related work, and Section 7 concludes.

Table 1: Representative single-transaction numeric logic error patterns from prior work, classified by bug abstraction level.

Pattern	Description
<i>Value-level: the wrong number flows through a computation.</i>	
Div In Path (Chen et al., 2025)	Division results used in conditions can lead to unexpected control flow due to integer truncation.
Exchange Problem (Chen et al., 2025)	Rounding errors in exchange calculations may result in zero output despite valid input, or receiving tokens without payment.
Precision Loss Trend (Chen et al., 2025)	Integer division’s inherent truncation tends to favor one side, potentially misaligning with protocol intent.
Minor Amount Retention (Chen et al., 2025)	Integer division in distributions leaves remainders unallocated to any recipient, causing permanent lock.
<i>Algorithm-level: operations are structurally arranged in a wrong way.</i>	
Operator Order Issue (Chen et al., 2025)	Arithmetic order matters: $a/b * c \neq a * c/b$ due to intermediate truncation.
Erroneous Accounting / Wrong Interest Rate Order (Sun et al., 2024; Zhang et al., 2023)	Balance calculations using outdated or prematurely updated interest rates yield incorrect results.
Erroneous Accounting / Wrong Checkpoint Order (Sun et al., 2024; Zhang et al., 2023)	Incorrect ordering between state updates and checkpoints leads to reward miscalculation.
Inconsistent State Updates (Zhang et al., 2023)	A required state variable update is omitted, causing subsequent calculations to use stale values (e.g., failing to accrue interest before a balance query).
Greedy Contract (Soud et al., 2024)	Contract can receive funds but lacks proper logic to withdraw them, causing permanent lock.

2 Background

2.1 Numeric Logic Errors in Solidity

Recent research has identified various numeric-related weaknesses in smart contracts (Chen et al., 2025; Sun et al., 2024; Soud et al., 2024; Zhang et al., 2023; Zhou et al., 2023). Building on these findings, numeric logic errors can be defined as follows:

Definition 1 (Numeric Logic Error) Bugs that allow transactions to complete successfully without compile-time or runtime exceptions, but violate the developer’s intended relationships among numerical values during function execution.

These errors come in two forms. Single-transaction errors occur within the execution of a single function call—where computation, function calls, and output generation happen in one atomic sequence. In contrast, multiple-transaction errors only emerge through sequences of function calls, requiring analysis of inter-function dependencies and contract-level invariants. Examples of multiple-transaction errors include Risky First Deposit and Price Oracle Manipulation (Sun et al., 2024; Zhang et al., 2023), which manifest only through state changes accumulated across multiple function invocations.

This paper focuses on single-transaction errors—those confined to one atomic execution, though possibly spanning multiple contracts through external calls. The choice aligns with our broader goal of a development-phase debugging assistant: a single-transaction error’s intended behavior is determined within the function the developer is currently writing, so the developer can declare it as input at the moment of writing. Multiple-transaction errors, whose intent spans sequences of calls authored at different times, are left to future work. Within this focus, this paper introduces a single organizing axis for the representative patterns from prior work: the bug’s abstraction level (Table 1). The axis recurs throughout the paper, later delineating IntentChecker’s design target (its intent model is deliberately confined to value-level relations; §3) and characterizing where the model reaches and where it does not (§4.2).

- **Value-level errors**—logic errors at the value level: a wrong operator, a swapped identifier, or truncation that produces a numerically incorrect result.
- **Algorithm-level errors**—logic errors in the structural arrangement of operations: operation ordering, an absent state update, or a missing procedure call.

The next subsection examines existing tools for numeric logic errors.

2.2 Structural Constraints of Existing Tools

Several tools target numeric logic errors. NumScout (Chen et al., 2025) catalogs five numerical defect patterns—Div In Path, Operator Order Issue, Exchange Problem, Precision Loss Trend, Minor Amount Retention—and detects them through LLM-pruned symbolic execution. GPTScan (Sun et al., 2024) targets logic vulnerabilities including several accounting-related patterns by combining GPT-based scenario matching with lightweight static analysis (Feist et al., 2019). ScType (Zhang, 2024) introduces a financial type system over Solidity variables—tuples of (financial type, scaling factor, token unit, address)—and detects accounting errors through type checking. These tools represent the current state of the art and have meaningfully advanced the field for auditing and testing workflows. However, on numeric logic errors specifically, these tools share two structural constraints, both rooted in the absence of any channel for per-case intent: who authors the detection criteria, and what input the analysis takes.

(1) Detection criteria: authored by the tool designer, not by the developer. Each tool matches the code against a fixed set of detection criteria authored at design time—pattern catalogs of known buggy shapes (Chen et al., 2025), financial type rules (Zhang, 2024), or LLM-driven matching against pre-defined vulnerability scenarios (Sun et al., 2024)—derived from prior bugs the tool designer has observed or catalogued. These criteria encode shapes of code that have been buggy in prior contracts, not what counts as a violation of the developer’s intent for this specific function. A bug is caught only when its surface form matches one of those prior shapes.

(2) Analysis phase: audit time, with the developer no longer in the loop. Existing numeric logic error detection tools take source code as their primary input and are positioned within auditing or testing workflows. They accept a contract (or a set of contracts), possibly augmented with type annotations or natural-language artifacts, and emit findings as a report. Because these tools run downstream of authoring, the

developer of the code under analysis is no longer available to be queried for clarification or to supply per-case input. The tool must therefore work entirely with the source code as written, and per-case intent has no channel into the analysis.

These two constraints are structural, not tool-specific. The above constraints are not characterized as failures of any specific tool, since the tools cited above are well-engineered within the design space they chose. The point is that the design space itself—post-hoc pattern-based detection without explicit intent input—requires intent to be inferred from code shape rather than supplied by the developer, which is a constraint on what can be expressed as input, not on how well the inference is implemented. This inference is reliable for vulnerability classes whose buggy form is invariant across contracts: a reentrancy pattern or an unchecked overflow is buggy regardless of what the developer intended, so the catalog’s implicit intent assumption holds. Numeric logic errors, by contrast, are intent-dependent by definition—the same arithmetic can be the correct formula in one contract (e.g., intentional floor division for fair-share rounding) and a silent bug in another (e.g., precision loss in token minting), distinguishable only by reference to what the developer intended this specific function to compute. A design space that excludes per-case intent has no signal on which to base that distinction. To step outside this constraint, the input must be enlarged (the developer must supply their intent) and the tool must be moved earlier in time (run during editing, when the developer is still available to supply that intent). These two moves are jointly necessary.

2.3 Proposed Overview

IntentChecker addresses each of these constraints with a dedicated component.

(1) Detection criteria: authored by the developer, via intent annotations.

IntentChecker introduces an intent model consisting of two annotation types—@During and @Post—that let developers declare numeric relationships at specific program points within a function and at function exit, together with a validation engine grounded in a denotational semantics that classifies each annotation as Satisfied, Warning, or Violated based on the intervals computed by abstract interpretation. The engine reports the judgment together with a risk score on a 0–10 scale and, when applicable, the sub-intervals in which the intent may be violated. Because the developer authors these annotations per case, the criteria are specific to the function being written rather than fixed to designer-anticipated shapes from prior contracts.

(2) Analysis phase: development time, with the developer in the loop.

For the annotations in (1) to be available at analysis time, IntentChecker is built on SolQDebug (Jeon et al., 2026), our prior work that performs statement-level interval-domain abstract interpretation inside the editor as code is being written. SolQDebug operates on a “range in, range out” principle: the developer supplies bounded ranges for entry-point values the function reads (transaction context, storage state, parameters) through debug annotations, and the analyzer reports the intervals each variable reaches at every program point, without itself judging whether those intervals correspond to correct or buggy behavior. To remain responsive as code is being written, it accepts partial code fragments through specialized grammar entry rules and maintains

an incremental control-flow graph that resumes analysis from the edit point rather than restarting on every change. This places IntentChecker in the same temporal slot as the developer who authors the function under analysis, opening the channel that (1) requires: per-case intent declared in the editor reaches the analysis directly, rather than being guessed from the source code after the fact.

Together, these two components position IntentChecker as an awareness tool rather than a detector: it does not pronounce “this is a bug” but instead brings the developer’s attention to the specific program points where the abstract execution cannot confirm the declared intent.

3 The Design of IntentChecker

3.1 System Architecture

Fig. 1 illustrates the overall architecture of IntentChecker.

Input. IntentChecker receives three types of input:

- **Source Fragment**—Solidity source fragments (block or statement) provided incrementally as the developer writes code, supported through SolQDebug’s partial-fragment grammar (§2.3).
- **Intent Annotation**—@During and @Post specifications expressing the developer’s intended numerical relationships.
- **Debug Annotation**—variable initialization directives that supply bounded value ranges at program points where the actual value is unknown to the analyzer. This category includes SolQDebug’s original @GlobalVar (transaction and block context values), @StateVar (state variables in contract storage), and @LocalVar (function parameters), as well as the @IReturn annotation introduced in this paper for interface call return values.

In addition, the Analysis Module loads pre-analyzed Dependency CFGs (.pkl files) for libraries and interfaces referenced from the target source file.

Output. For each intent annotation, IntentChecker produces:

- **Result**—a validation state of Satisfied, Violated, or Warning.
- **Risk Score**—a 0–10 scale score: 0.0 for Satisfied, 10.0 for Violated, and 0.1–9.9 for Warning reflecting the severity of the potential violation.
- **Violated Region**—the specific sub-intervals where the intent may be violated, helping developers identify problematic input ranges.

Internal Modules. Phases 1 and 2 reuse and extend SolQDebug’s analysis pipeline to accommodate the new @IReturn annotation and the multi-contract deployment-unit scope, while Phase 3 is IntentChecker’s core contribution.

- **Phase 1 — Parsing Module.** Uses ANTLR for syntax analysis of both Solidity source fragments and intent/debug annotation comments, and additionally delegates the Solidity source fragments to the official Solidity compiler for semantic validation.
- **Phase 2 — Analysis Module.** Performs incremental abstract interpretation over the interval domain, updating the control-flow graph and variable states as each

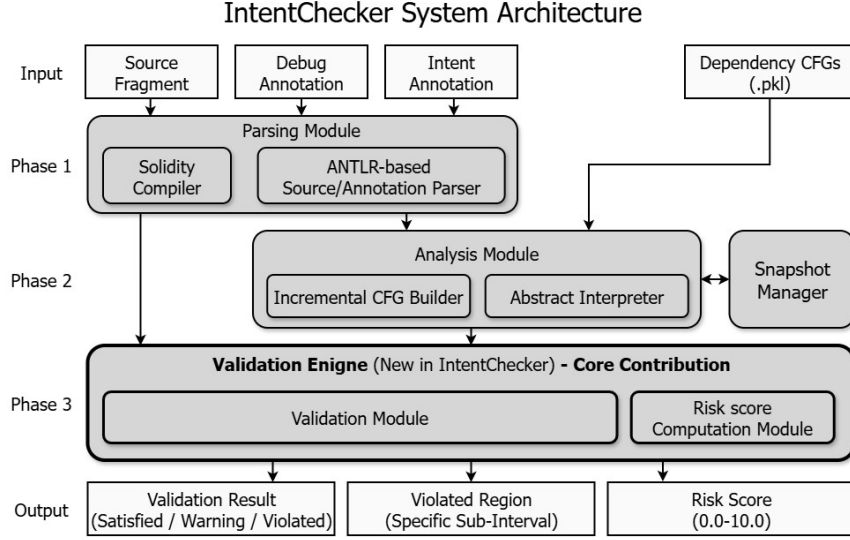


Fig. 1: System architecture of IntentChecker.

statement is processed. When a debug annotation is received, the Snapshot Manager (reused unchanged from SolQDebug) preserves the current analysis state, allowing the analyzer to overwrite the targeted variable’s interval with the developer-supplied range, re-analyze affected code paths, and restore the pre-annotation state afterwards; the same mechanism handles the new `@IReturn` annotation, since injecting a range at an interface call return site is mechanically the same as injecting one at a `@StateVar` declaration. The module additionally supports inline assembly written in Yul for arithmetic computation, leaving non-arithmetic constructs at $\top$.

- **Phase 3 — Validation Engine.** Draws on two signals from the earlier phases—parsed intent annotations from the Parsing Module, which encode the relational constraints declared in `@During/@Post` annotations, and computed variable intervals from the Analysis Module, which represent the abstract state of each variable at the annotated program point. For each annotation, it evaluates whether the declared intent holds by comparing the actual intervals against the specified constraints, producing the three output fields listed above.

Analysis scope. IntentChecker analyzes one function at a time within a multi-contract scope, extending SolQDebug’s (Jeon et al., 2026) original single-contract analysis to cover libraries and inherited parents that share storage with the target contract. External contracts—separately deployed and reached through an address or


```

1 // contract state: bool presell; uint256 ethRateFix;
2 // helper: function calculateRate() public view returns (uint256);
3 function buy() public payable {
4     // @GlobalVar msg.value = [50000000000500000, ...]
5     // @StateVar presell = true
6     require(presell, "presell_is_closed");
7     uint256 amountToBuy = msg.value / ethRateFix * calculateRate();
8     // @During amountToBuy * ethRateFix >= msg.value * 250000
9     balances[msg.sender] += amountToBuy;
10 }

```

Fig. 2: Operator order issue in exchange rate calculation (simplified).

interface-typed variable—sit outside this scope, because their storage resides at a different address that the target’s annotations cannot reach. This exclusion creates a problem for analysis: any value derived from an interface call resolves to $\top$ and propagates into every downstream computation, defeating intent validation for functions that read oracles or external token state. To address this, `@IReturn` extends the same range-injection mechanism the three existing annotations rely on: the developer supplies a bounded range for a value the function reads whose origin lies outside the analyzer’s view, and the analyzer proceeds under it. Interface functions split into two cases: those declared `view` or `pure`, where Solidity guarantees no state-modifying side effects, and those that may modify state—potentially even re-entering the target function. Injection is restricted to the former, allowing intent validation to proceed under the developer-supplied range rather than being defeated by $\top$ propagation.

3.2 Examples of Intent Annotation Usage

This section walks through two examples to illustrate how an annotated function produces a validation judgment. The first case demonstrates a mid-execution check via a `@During` annotation; the second demonstrates a post-execution check via `@Post` together with an `@IReturn` annotation for external contract state.

Case 1: `@During` — Operator Order Issue. Fig. 2 shows a token purchase function where operator precedence causes the buyer to receive fewer tokens than the exchange rate warrants.

The developer’s intent is that `amountToBuy` should satisfy the exchange-rate relation—the tokens received, times the rate denominator, should be at least the ETH paid times the rate numerator. Because `msg.value / ethRateFix` truncates before multiplication, this intent is violated for non-round input amounts. The `@During` annotation (line 8) captures this intent directly, and `IntentChecker` reports `Violated`.

Case 2: `@Post` — Erroneous Accounting. Fig. 3 shows a rebasing vault’s mint function. The bug is that minted proxy tokens are computed from the total vault balance (including the just-deposited amount), which inflates the shares beyond the deposited amount when the redeem rate deviates from 1.

The `@Post` `_balances[to] ≤ amount` annotation (line 16) expresses the intent that no user should receive more shares than the tokens they deposited. The `@IReturn` debug annotation (lines 8–9) injects the range `[1500e18, 1500e18]` for the external `balanceOf` call (line 14), which the analyzer would otherwise evaluate as $\top$. The

```

1 // contract state: uint256 _totalSupply; mapping(address => uint256) _balances;
2 // address baseToken; uint256 ONE; function redeemRate() public view returns (uint256);
3 function mint(address to, uint256 amount)
4     public override returns (uint256) {
5     // @LocalVar amount = [500e18, 500e18]
6     // @StateVar _totalSupply = [1000e18, 1000e18]
7     // @StateVar _balances[to] = [0, 0]
8     // @IReturn IERC20(baseToken).balanceOf()
9     //     = [1500e18, 1500e18]
10    uint256 _redeemRate = redeemRate();
11    require(IERC20(baseToken).transferFrom(
12        msg.sender, address(this), amount));
13    uint256 baseBalance =
14        IERC20(baseToken).balanceOf(address(this));
15    uint256 proxy = (baseBalance * ONE) / _redeemRate;
16    // @Post _balances[to] <= amount
17    _mint(to, proxy);
18 }

```

Fig. 3: Erroneous accounting in vault mint with inflated proxy amount (simplified).

transferFrom (line 11) is state-modifying and therefore cannot carry an @IReturn annotation, per the view/pure restriction described in §3. With these annotations, IntentChecker computes the proxy amount and finds that it exceeds amount, reporting Violated. This example demonstrates how @IReturn enables intent validation for functions that depend on external contract state.

These two examples illustrate the two temporal scopes of intent validation—mid-execution checks via @During and post-execution checks via @Post. The complete intent model supporting these annotations is defined in the following section.

3.3 Intent Model

The denotational semantics of the intent model is built around a meaning function

$$\mathcal{V}[\cdot] : \mathbb{A} \rightarrow (\mathbb{C} \rightarrow \mathbb{J}),$$

which assigns to each intent annotation $a \in \mathbb{A}$ a partial function from validation contexts to tri-valued judgments. The model consists of three components: the syntactic domain $\mathbb{A}$, the semantic domain $\mathbb{C}$, and the judgment set $\mathbb{J}$.

Syntactic domain. $\mathbb{A}$ is the set of intent annotations generated by the grammar in §3.3.1, with a typical annotation denoted by a . Each annotation begins with either @During or @Post and contains one atomic intent clause.

The grammar distinguishes three classes of forms:

- **@During-specific forms**, attached to an individual program statement and evaluated at the corresponding program point;
- **@Post forms**, evaluated with respect to the function-exit context; and
- **Common forms**, which may be used under either @During or @Post, with their reference environment determined by the annotation kind.

Table 2: Fields of the validation context $\Gamma \in \mathbb{C}$. Each annotation form reads only the fields required by its semantics.

Field	Meaning
$\sigma_{\text{pt}} \in \Sigma$	abstract state at the annotated program point
$\sigma_{\text{before}} \in \Sigma$	state immediately before the statement at the annotated line
$\sigma_{\text{assign}} \in \Sigma$	state at the relevant variable-assignment point within the enclosing function
$\sigma_{\text{entry}} \in \Sigma$	state at function entry, after parameter binding and supplied analysis inputs
$\sigma_{\text{exit}} \in \Sigma$	joined state at normal function exit
$R : \text{Line} \rightarrow \text{Val}^*$	partial map from return-statement lines to captured return values
$\mathcal{K}$	statements associated with the annotated source location, used for call-argument and require/assert lookup
$\Pi \subseteq \Sigma$	reachable abstract states accumulated up to the annotation’s evaluation boundary

Each annotation expresses one atomic numeric obligation. When a reported finding requires multiple independently checkable obligations, we represent them as separate annotations rather than combining them with annotation-level logical connectives.

Semantic domain. We use two semantic domains:

- Σ — the space of abstract variable environments produced by the interval-domain interpreter. Each $\sigma \in \Sigma$ maps program variables to interval-valued abstract states, including compound values such as arrays, mappings, and structs whose numeric leaves are represented by intervals.
- $\mathbb{C}$ — the set of validation contexts. Each $\Gamma \in \mathbb{C}$ is a record containing the program-state snapshots and auxiliary data available when an annotation is evaluated.

Field supply. The fields available in Γ depend on the annotation kind:

- **@During:** σ_{pt} , σ_{before} , σ_{assign} , σ_{entry} , R , $\mathcal{K}$, and Π , where Π contains reachable states from function entry up to the annotated program point;
- **@Post:** σ_{entry} , σ_{exit} , R , and Π , where Π contains reachable states from function entry to normal exit.

This distinction also determines the available snapshot qualifiers: **before**, **after**, and **assign** are exposed only under **@During**, whereas **entry** and **exit** are exposed only under **@Post**. Unqualified references remain valid in both contexts.

Judgment set. $\mathbb{J} = \{S, V, W\}$ is the set of tri-valued judgments:

- **Satisfied** (S) — every concrete state represented by the abstract state satisfies the annotation;
- **Violated** (V) — every represented concrete state violates the annotation; and
- **Warning** (W) — the abstract state is not precise enough to establish either satisfaction or violation.

duringIntent	→	// @During duringClause
postIntent	→	// @Post postClause
duringClause	→	identifier .arg[n] relOp intentValue _{D}
		require feasible
		assert feasible
		commonClause _{D}
postClause	→	commonClause _{P}
commonClause _{k}	→	returnExpression relOp intentValue _{k}
		return[n] relOp intentValue _{k}
		intentValue _{k} relOp PercentOf(intentValue _{k} , p)
		intentValue _{k} relOp ceil(intentValue _{k} , d)
		intentValue _{k} relOp floor(intentValue _{k} , d)
		intentValue _{k} relOp intentValue _{k}
		intentValue _{k} => intentValue _{k}
		changed(intentValue _{k} , true false)
intentValue _{k}	→	arithExpr _{k}
arithExpr _{k}	→	arithExpr _{k} (<< >> >>>) arithAdd _{k} arithAdd _{k}
arithAdd _{k}	→	arithAdd _{k} (+ -) arithTerm _{k} arithTerm _{k}
arithTerm _{k}	→	arithTerm _{k} (* / %) arithExp _{k} arithExp _{k}
arithExp _{k}	→	arithFactor _{k} ** arithExp _{k} arithFactor _{k}
arithFactor _{D}	→	number [number , number] varRef varRef(before) varRef(after) varRef(assign) (arithFactor _{D})
arithFactor _{P}	→	number [number , number] varRef varRef(entry) varRef(exit) (arithExpr _{P})
varRef	→	identifier subAccess*
subAccess	→	. identifier [expr]
relOp	→	< > ≤ ≥ == != in not in

Fig. 4: Grammar of intent annotations. Subscripts D and P denote expression contexts under **@During** and **@Post**, respectively. Snapshot-qualified references are context-sensitive: **before**, **after**, and **assign** are available only under **@During**, whereas **entry** and **exit** are available only under **@Post**. Sub-items are tagged with rule labels ($D_{\bullet}$), ($P_{\bullet}$), and ($C_{\bullet}$) for reference in the text.

For an annotation $a \in \mathbb{A}$ and validation context $\Gamma \in \mathbb{C}$, **IntentChecker** reports $\mathcal{V}[a](\Gamma) \in \mathbb{J}$. The partiality of $\mathcal{V}[a]$ captures that the meaning function is defined only for well-formed contexts that provide all fields required by the selected annotation form.

The remainder of this section proceeds as follows. Section 3.3.1 defines the syntax of $\mathbb{A}$ and the denotational semantics of its annotation forms. Section 3.3.2 then presents the executable validation procedures that implement these semantics.

3.3.1 Syntax and Semantics

Grammar overview. Fig. 4 gives the syntax of intent annotations, which are embedded in single-line comments so that they coexist with Solidity code without affecting compilation. Intent-value expressions support arithmetic over variable references with

member and index access, numeric literals, inline intervals $[a, b]$, and the DSL helpers `PercentOf`, `ceil`, and `floor`.

Snapshot-qualified references are context-sensitive. Within a `@During` intent, a variable reference may be qualified with `before`, `after`, or `assign`, referring to the statement-level snapshots defined in Table 2. Within a `@Post` intent, a variable reference may instead be qualified with `entry` or `exit`, referring to the function-entry and function-exit snapshots. An unqualified variable reference is valid in both contexts and is evaluated against the ambient reference environment $\text{ref}(\Gamma)$ defined below. Relational operators include $<$, $>$, $\leq$, $\geq$, $==$, $!=$, and the membership operators `in` and `not in`; the implication operator $\Rightarrow$ is a truthiness-based atomic form whose semantics is defined below.

Well-formedness. We write $\Gamma \vdash a : \mathbf{wf}$ to mean that annotation a is well-formed under Γ : a conforms to Fig. 4, every variable reference in a resolves in the relevant field of Γ , and all form-specific auxiliary data required by a is available. The equations below define $\mathcal{V}[[a]](\Gamma)$ only for well-formed inputs; annotation forms whose required auxiliary data is missing correspond to implementation error paths and lie outside the semantic claim.

Well-formedness is orthogonal to the informativeness of the abstract inputs. Debug annotations inherited from `SolQDebug`, such as `@StateVar`, `@LocalVar`, and `@IReturn`, may be used to provide bounded intervals for otherwise-unknown values. Without such constraints, some inputs may remain $\top$, in which case the primitives defined below may yield W even though the annotation itself is well-formed.

Expression evaluation. Let $\mathcal{I}$ denote the set of integer intervals. Intent-value expressions are evaluated against an abstract variable environment $\sigma \in \Sigma$ by

$$\text{eval} : \Sigma \times \text{intentValue} \rightarrow \mathcal{I}.$$

The evaluator delegates ordinary arithmetic subexpressions to the underlying interpreter and handles annotation-specific constructs such as `PercentOf`, `ceil`, `floor`, and inline interval literals $[a, b]$. A numeric literal n evaluates to the singleton interval $[n, n]$.

Rules whose left-hand sides are not themselves intent-value expressions are handled separately. For rule (D_{arg}) , $\text{arg}_n(f, \mathcal{K})$ resolves the n -th argument expression of the annotated call and passes it to `eval`. Rules (C_{ret}) and (C_{ret_n}) instead access the return-summary field R directly as $R(\ell_{\text{ret}})$ and $R(\ell_{\text{ret}})[n]$.

Snapshot-qualified references. A snapshot-qualified reference overrides the ambient environment and is evaluated against the corresponding stored snapshot:

$$\begin{aligned} \text{eval}(\sigma, e(\text{entry})) &= \text{eval}(\sigma_{\text{entry}}, e), & \text{eval}(\sigma, e(\text{exit})) &= \text{eval}(\sigma_{\text{exit}}, e), \\ \text{eval}(\sigma, e(\text{before})) &= \text{eval}(\sigma_{\text{before}}, e), & \text{eval}(\sigma, e(\text{after})) &= \text{eval}(\sigma_{\text{pt}}, e), \\ & \text{eval}(\sigma, e(\text{assign})) &= \text{eval}(\sigma_{\text{assign}}, e). \end{aligned}$$

Because the qualifier selects the snapshot locally for the referenced value, snapshot-qualified references may appear anywhere inside an arithmetic expression. For

example,

$$debts == debts(entry) + increasingDebt$$

is represented as an ordinary relational comparison: the unqualified occurrence of *debts* is evaluated in $\text{ref}(\Gamma)$, while *debts(entry)* is evaluated in σ_{entry} . Consequently, entry/exit and before/after comparisons do not require separate clause-level grammar rules; they are ordinary instances of (C_{cmp}) .

Feasibility evaluator. To support the feasibility forms, we use the space of abstract boolean intervals

$$\mathcal{B} = \{\{0\}, \{1\}, \{0, 1\}\}$$

and a boolean-expression evaluator

$$\text{eval}_{\mathcal{B}} : \Sigma \times \text{BExpr} \rightarrow \mathcal{B},$$

where BExpr denotes the base Solidity boolean-expression category. The operators $\text{cond}_{\text{req}}(\mathcal{K})$ and $\text{cond}_{\text{asrt}}(\mathcal{K})$ extract the conditions of the annotated **require** and **assert** statements, respectively.

Comparison primitive. Relational annotations are grounded in

$$\text{Cmp}^* : \mathcal{I} \times \text{Op} \times \mathcal{I} \rightarrow \mathbb{J},$$

defined by

$$\text{Cmp}^*(L, op, R) = \begin{cases} S & \text{if } \forall l \in L, \forall r \in R : l \text{ op } r, \\ V & \text{if } \forall l \in L, \forall r \in R : \neg(l \text{ op } r), \\ W & \text{otherwise.} \end{cases}$$

For $op \in \{\text{in}, \text{not in}\}$, set membership replaces the pointwise relation.

Truthiness implication primitive. The implication form $\Rightarrow$ is interpreted over the zero/non-zero status of the two interval operands:

$$\text{Impl}^* : \mathcal{I} \times \mathcal{I} \rightarrow \mathbb{J},$$

with

$$\text{Impl}^*(L, R) = \begin{cases} S & \text{if } L = \{0\}, \\ S & \text{if } 0 \notin L \text{ and } 0 \notin R, \\ V & \text{if } 0 \notin L \text{ and } R = \{0\}, \\ W & \text{otherwise.} \end{cases}$$

Feasibility primitive. The feasibility forms use

$$\text{Feasible}^* : \mathcal{B} \rightarrow \mathbb{J}, \quad \text{Feasible}^*(B) = \begin{cases} V & \text{if } B = \{0\}, \\ S & \text{otherwise.} \end{cases}$$

Thus a condition is violated only when it is provably false for every concrete state represented by the abstract boolean interval.

Meaning function. The meaning function $\mathcal{V}[\cdot]$, introduced in §3.3, is defined by case analysis over the annotation forms. We use e for an intent-value expression, f for a function identifier, and ℓ_{ret} for the return-statement line selected by the annotation.

@During annotations. The forms specific to @During are argument comparison and feasibility. Let $\mathcal{K}$ denote the statements at the annotated source location. The n -th call argument is evaluated immediately before the call, while feasibility is evaluated at the annotated program point:

$$\begin{aligned}\mathcal{V}[f.\text{arg}[n] \text{ op } e](\Gamma) &= \text{Cmp}^*(\text{eval}(\sigma_{\text{before}}, \text{arg}_n(f, \mathcal{K})), \text{op}, \text{eval}(\sigma_{\text{before}}, e)), \\ \mathcal{V}[\text{require feasible}](\Gamma) &= \text{Feasible}^*(\text{eval}_{\mathcal{B}}(\sigma_{\text{pt}}, \text{cond}_{\text{req}}(\mathcal{K}))), \\ \mathcal{V}[\text{assert feasible}](\Gamma) &= \text{Feasible}^*(\text{eval}_{\mathcal{B}}(\sigma_{\text{pt}}, \text{cond}_{\text{asrt}}(\mathcal{K}))).\end{aligned}$$

All @Post forms are common forms and therefore require no additional annotation-specific equations.

Common-form annotations. For forms that may appear under either annotation kind, let

$$\text{ref}(\Gamma) = \begin{cases} \sigma_{\text{pt}} & \text{under @During,} \\ \sigma_{\text{exit}} & \text{under @Post.} \end{cases}$$

The common forms are then defined uniformly as

$$\begin{aligned}\mathcal{V}[\text{returnExpression op } e](\Gamma) &= \text{Cmp}^*(R(\ell_{\text{ret}}), \text{op}, \text{eval}(\text{ref}(\Gamma), e)), \\ \mathcal{V}[\text{return}[n] \text{ op } e](\Gamma) &= \text{Cmp}^*(R(\ell_{\text{ret}})[n], \text{op}, \text{eval}(\text{ref}(\Gamma), e)), \\ \mathcal{V}[e_l \text{ op } e_r](\Gamma) &= \text{Cmp}^*(\text{eval}(\text{ref}(\Gamma), e_l), \text{op}, \text{eval}(\text{ref}(\Gamma), e_r)), \\ \mathcal{V}[e_l \Rightarrow e_r](\Gamma) &= \text{Impl}^*(\text{eval}(\text{ref}(\Gamma), e_l), \text{eval}(\text{ref}(\Gamma), e_r)).\end{aligned}$$

The change-detection form $\text{changed}(e, b)$ compares the value of e at function entry with reachable abstract states in the path-state set Π . For $\text{changed}(e, \text{true})$:

$$\mathcal{V}[\text{changed}(e, \text{true})](\Gamma) = \begin{cases} S & \text{if } \exists \sigma \in \Pi : \text{Cmp}^*(\text{eval}(\sigma_{\text{entry}}, e), =, \text{eval}(\sigma, e)) = V, \\ V & \text{if } \forall \sigma \in \Pi : \text{Cmp}^*(\text{eval}(\sigma_{\text{entry}}, e), =, \text{eval}(\sigma, e)) = S, \\ W & \text{otherwise.} \end{cases}$$

The dual form $\text{changed}(e, \text{false})$ requires equality with the entry value throughout the relevant reachable states:

$$\mathcal{V}[\text{changed}(e, \text{false})](\Gamma) = \begin{cases} S & \text{if } \forall \sigma \in \Pi : \text{Cmp}^*(\text{eval}(\sigma_{\text{entry}}, e), =, \text{eval}(\sigma, e)) = S, \\ V & \text{if } \exists \sigma \in \Pi : \text{Cmp}^*(\text{eval}(\sigma_{\text{entry}}, e), =, \text{eval}(\sigma, e)) = V, \\ W & \text{otherwise.} \end{cases}$$

The set Π is populated according to the annotation context: under @During, it contains reachable states from function entry up to the annotated program point;

Algorithm 1 Auxiliary Outputs Computation for Cmp^* -based Annotations

```
1: Input: Intervals  $L, R$ , operator  $op$ , state  $j \in \mathbb{J}$ 
2: Output: ( $risk\_score, regions$ ) with  $risk\_score \in [0.0, 10.0]$  and  $regions =$ 
   ( $L_{lower}, L_{upper}, L_{overlap}$ )
3:  $(|T_s|, |T_v|, |T_w|) \leftarrow RegionSizes(L, R, op)$ 
4:  $regions \leftarrow ExtractViolatedRegion(L, R, op)$ 
5: if  $j = satisfied$  then
6:   return ( $0.0, regions$ )
7: else if  $j = violated$  then
8:   return ( $10.0, regions$ )
9: end if
10:  $p \leftarrow p_{true}(L, R, op)$ 
11:  $risk\_type \leftarrow ClassifyRisk(regions, op)$ 
12:  $(base, span) \leftarrow BandOf(risk\_type)$ 
13:  $risk\_score \leftarrow base + (1 - p) \times span$ 
14: return ( $risk\_score, regions$ )
```

under @Post , it contains reachable states from function entry to function exit. Thus the same pair of equations defines change detection for both annotation kinds.

3.3.2 Auxiliary Outputs: Violated Region and Risk Score

This subsection presents the executable procedures that accompany the semantic realisation of §3.3.1, producing two auxiliary outputs derived from the same interval analysis that drives Cmp^* :

- **violated region** — sub-intervals of the input range L that witness the failure of the comparison;
- **risk score** — a value on a 0–10 scale combining the state, the true probability, and the shape of the violated region.

Both outputs are downstream of the semantic claim and do not refine the judgment $j \in \mathbb{J}$; they support warning ranking and developer feedback. An internal probability $p_{true}(L, R, op) \in [0, 1]$ — the chance that the comparison holds under a uniform distribution over $L \times R$ — mediates the risk-score computation but is not surfaced separately. Impl^* and Feasible^* are directly computable from their definitions and need no dedicated algorithm, and the change-detection form iterates Π directly. The remaining auxiliary outputs for Cmp^* -based annotations are computed by Algorithm 1 in four steps, each elaborated in turn below and cited with its line range in the algorithm.

Step 1: Region partitioning (Algorithm 1, line 3). Given integer intervals $L = [l_{\min}, l_{\max}]$ and $R = [r_{\min}, r_{\max}]$ with an operator op , the interpreter partitions L into three disjoint regions:

- T_s (satisfied region): values of L for which $l \text{ op } r$ holds for all $r \in R$;
- T_v (violated region): values of L for which $l \text{ op } r$ fails for all $r \in R$;
- T_w (warning region): values of L whose truth value depends on the actual value of R .

Table 3: Per-operator satisfied/violated conditions and region sizes. All sizes are clamped to $\max(0, \cdot)$.

op	Satisfied condition	Violated condition	$ T_s $	$ T_v $
$>$	$l_{\min} > r_{\max}$	$l_{\max} \leq r_{\min}$	$l_{\max} - r_{\max}$	$r_{\min} - l_{\min} + 1$
$\geq$	$l_{\min} \geq r_{\max}$	$l_{\max} < r_{\min}$	$l_{\max} - r_{\max} + 1$	$r_{\min} - l_{\min}$
$<$	$l_{\max} < r_{\min}$	$l_{\min} \geq r_{\max}$	$r_{\min} - l_{\min}$	$l_{\max} - r_{\max} + 1$
$\leq$	$l_{\max} \leq r_{\min}$	$l_{\min} > r_{\max}$	$r_{\min} - l_{\min} + 1$	$l_{\max} - r_{\max}$
$=$	$L = R$, both singletons	$L \cap R = \emptyset$	$ L \cap R $	$n_L - L \cap R $
$\neq$	$L \cap R = \emptyset$	$L = R$, both singletons	$n_L - L \cap R $	$ L \cap R $
in	$L \subseteq R$	$L \cap R = \emptyset$	$ L \cap R $	$n_L - L \cap R $
not in	$L \cap R = \emptyset$	$L \subseteq R$	$n_L - L \cap R $	$ L \cap R $

Table 3 gives the satisfied/violated conditions and region sizes per operator, where $n_L = l_{\max} - l_{\min} + 1$ and $n_R = r_{\max} - r_{\min} + 1$ denote the number of integer values in each interval. The state j returned by Cmp^* follows directly: $j = S$ when the satisfied condition holds, $j = V$ when the violated condition holds, and $j = W$ when neither holds. Fig. 5 illustrates this partition for $op = ">"$ across the spectrum of L vs R relative positions on the integer line: from L entirely below R (Verdict: Violated) through several warning configurations to L entirely above R (Verdict: Satisfied). The same Table 3 formulas apply throughout, with $\max(0, \cdot)$ clamping handling the empty regions in each configuration.

Step 2: Violated-region extraction (Algorithm 1, line 4). The interpreter also extracts the violated region of L : sub-intervals of L that witness failure. Table 4 decomposes L into three directional components per operator:

- L_{lower} — values of L that fall below the acceptable range of R ;
- L_{upper} — values of L that exceed the acceptable range of R ;
- L_{overlap} — values of L whose outcome depends on the concrete R -value (the warning region for ordering operators; the coincidence zone for equality/membership).

Each component is an integer sub-interval or empty; empty components indicate that no witness exists in that direction. For the $=$ operator, when L and R are disjoint, the overlap is empty and L falls entirely into exactly one of L_{lower} or L_{upper} depending on which side it lies on. The risk-classification step below consumes this decomposition, and the same sub-intervals are surfaced to the developer as the concrete ranges where the annotation may fail.

Step 3: State-based scoring (Algorithm 1, lines 5–9). Lines 5–9 map the state j returned by Cmp^* directly to extreme scores: $j = \text{satisfied}$ returns 0.0 (fully satisfied condition) and $j = \text{violated}$ returns 10.0 (definite violation). The warning case $j = W$ requires further composition and is handled in Step 4.

Step 4: Warning-case scoring (Algorithm 1, lines 10–13). When $j = W$, the risk score is composed from an internal probability p_{true} and a risk-type classification

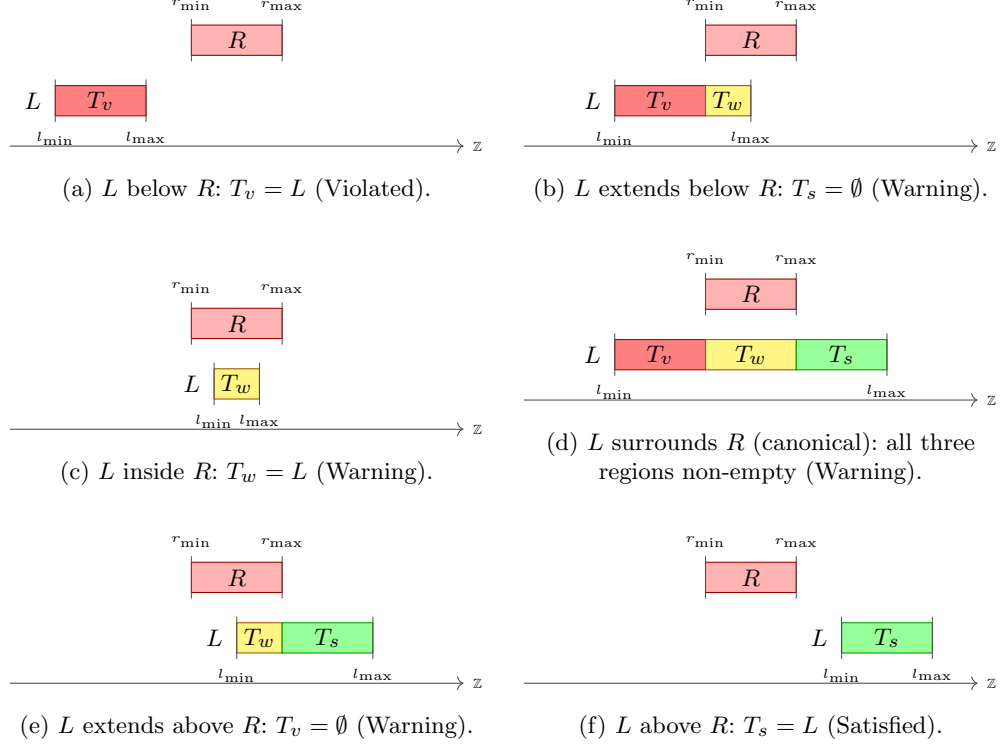


Fig. 5: Spectrum of L vs R configurations for $op = ">"$.

Table 4: Violated-region extraction formulas. Endpoints are clamped so that $\min > \max$ yields empty; “—” denotes no witness in that direction.

op	L_{lower}	L_{upper}	$L_{overlap}$
$>$	$[l_{\min}, \min(l_{\max}, r_{\min})]$	—	$[\max(l_{\min}, r_{\min}+1), \min(l_{\max}, r_{\max})]$
$\geq$	$[l_{\min}, \min(l_{\max}, r_{\min}-1)]$	—	$[\max(l_{\min}, r_{\min}), \min(l_{\max}, r_{\max})]$
$<$	—	$[\max(l_{\min}, r_{\max}), l_{\max}]$	$[\max(l_{\min}, r_{\min}), \min(l_{\max}, r_{\max}-1)]$
$\leq$	—	$[\max(l_{\min}, r_{\max}+1), l_{\max}]$	$[\max(l_{\min}, r_{\min}), \min(l_{\max}, r_{\max})]$
$=$	$[l_{\min}, (L \cap R)_{\min}-1]$	$[(L \cap R)_{\max}+1, l_{\max}]$	$L \cap R$
$\neq$	—	—	$L \cap R$
in	$[l_{\min}, \min(l_{\max}, r_{\min}-1)]$	$[\max(l_{\min}, r_{\max}+1), l_{\max}]$	—
not in	—	—	$L \cap R$

over the violated-region components, then assembled into a value in $(0.0, 10.0)$ by a banded linear formula.

True probability (Algorithm 1, line 10). The true probability $p_{true}(L, R, op) \in [0, 1]$ that the comparison holds under a uniform distribution over

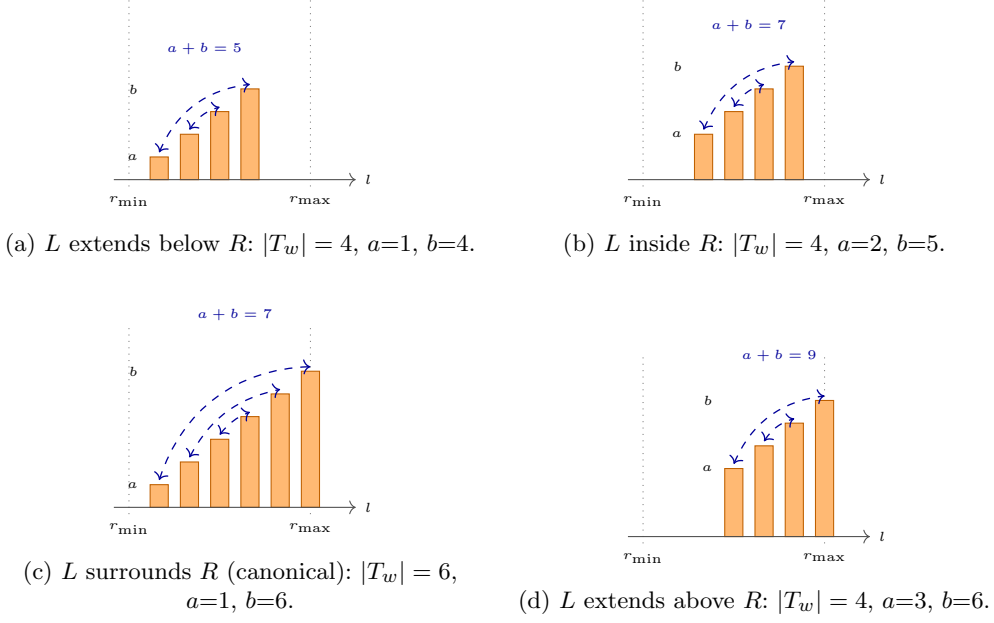


Fig. 6: Arithmetic-sequence visualization of E_w across warning configurations for $op = ">"$ with $n_R = 7$.

$L \times R$ is defined in closed form using the region sizes from Step 1:

$$p_{true}(L, R, op) = \begin{cases} |T_s|/n_L & \text{if } op \in \{\text{in, not in}\}, \\ |T_s|/(n_L \cdot n_R) & \text{if } op = "=" \text{ and } n_R > 1, \\ 1 - |T_v|/(n_L \cdot n_R) & \text{if } op = "\neq" \text{ and } n_R > 1, \\ (|T_s| + E_w)/n_L & \text{otherwise.} \end{cases}$$

- For $op \in \{\text{in, not in}\}$, $|T_s|$ already counts the L -values inside R , yielding $|T_s|/n_L$ directly.
- For $op = "="$ or $\neq$ with $n_R > 1$, each overlapping L -value has a $1/n_R$ probability of matching any particular R -value, so satisfying pairs are counted over the product space and divided by $n_L \cdot n_R$ (all pairs).
- Otherwise (when $op \in \{>, \geq, <, \leq\}$, or $=/\neq$ with $n_R = 1$), the warning region T_w is the range of L -values whose outcome depends on R . Since Solidity operates on integers, for each $l \in T_w$ the number of R -values satisfying $l op r$ forms an arithmetic sequence across T_w . Fig. 6 visualizes this sequence across the four warning configurations identified in Fig. 5; in each panel, pairing symmetric terms (first with last, second with second-to-last, and so on) yields a constant pair-sum equal to $a+b$,

so the $|T_w|$ counts sum to $(a + b)|T_w|/2$, and dividing by n_R gives

$$E_w = \frac{(a + b)|T_w|}{2n_R},$$

where a and b are the numbers of satisfying R -values at the smallest and largest elements of T_w , respectively. When $n_R = 1$ for $=$ or $\neq$, $E_w = 0$ and the formula degenerates to pointwise evaluation.

Risk type classification (Algorithm 1, line 11). For warning results, the engine classifies the violation pattern into three risk types based on which components of Table 4 are non-empty:

- **Type 1** — no directional witness in L_{lower} or L_{upper} (only an overlap zone, representing a mild uncertainty);
- **Type 2** — exactly one of L_{lower} or L_{upper} is non-empty (a single-direction violation);
- **Type 3** — both L_{lower} and L_{upper} are non-empty, or the operator is $=/\neq$ (whose interval semantics is inherently imprecise and is assigned the highest band by convention).

Banded scoring (Algorithm 1, lines 12–13). Each type maps to a fixed $(base, span)$ band — $(0.1, 3.3)$ for type 1, $(3.4, 6.6)$ for type 2, $(6.7, 9.9)$ for type 3 — and the final score within the band is $base + (1 - p_{true}) \times span$, so that a lower p_{true} (a more probable violation) yields a higher score.

This banded structure is designed for at-a-glance interpretability: 0 marks safe execution, 10 a certain violation, and the three bands group warnings by structural severity—mild overlap-only ambiguity (Type 1), single-direction violation (Type 2), and full bilateral or imprecise comparison (Type 3). Developers can therefore prioritize reviews directly by score without inspecting the underlying probability or region structure.

This completes the design of IntentChecker’s intent model and validation engine. The full implementation, together with the evaluation benchmark and scripts used in the following section, is publicly available (IntentChecker, 2026a).

4 Evaluation

We evaluate IntentChecker as an intent-based approach for expressing and validating numeric intent associated with numeric logic errors through three research questions examining its capabilities, associated costs, and position within the broader design space of numeric error analysis.

- **RQ1 (Expressibility and Validatability):** To what extent can the numeric intent associated with reported numeric logic errors be expressed and validated by IntentChecker?

- **RQ2 (Specification Requirements and Analysis Costs):** What specification requirements and analysis costs are associated with specifying and validating numeric intent?
- **RQ3 (Comparison with Existing Detection Tools):** How does IntentChecker compare with existing numeric logic error detection tools in terms of analysis model, coverage, and latency?

We first describe the implementation environment and benchmark dataset shared across the three research questions.

4.1 Experimental Setup

Experimental Setting. The evaluation environment consists of an 11th Gen Intel® Core™ i7-11390H CPU at 3.40 GHz with 16.0 GB RAM, running Windows 10 (64-bit).

Implementation and Execution. IntentChecker is implemented in Python 3.10 and operates directly on Solidity source code without requiring compilation or deployment.

For the evaluation, IntentChecker is executed in a controlled scripting environment that supplies source-code and annotation events to the interpreter in sequence. Source-code events incrementally update the program representation, and when a source fragment is added or modified, analysis is resumed from the affected source location rather than recomputing the preceding program state. Intent annotations register the properties to be validated at their corresponding source locations.

To provide the symbolic inputs required for analysis, IntentChecker uses the batch-annotation mechanism introduced in SolQDebug (Jeon et al., 2026). Once a complete batch annotation is provided, the analyzer evaluates the registered intents under the specified abstract state.

Dataset Collection. We initially collect 89 real-world numeric logic error instances from three sources covering a range of numeric error patterns. We then apply the exclusion criteria described below to obtain a final benchmark of 74 eligible instances.

NumScout (7 instances). NumScout (Chen et al., 2025) provides a curated collection of numeric logic errors mined from deployed BNB Smart Chain contracts. From the 95-sample subset distributed in NumScout’s official repository (NumScout, 2026a), we primarily restrict the selection to contracts targeting Solidity ≥ 0.6 , the version range supported by IntentChecker’s front-end. We retain one contract declaring a 0.4.x pragma because the constructs exercised by the evaluated case are nevertheless supported by the current front-end without modification. Among the compatible cases, we select seven instances covering all five numerical defect patterns enumerated in NumScout: one Div In Path, two Exchange Problem, one Operator Order Issue, two Precision Loss Trend, and one Minor Amount Retention instance. One of the Exchange Problem instances is labeled Profit Opportunity in the repository.

Flyinointment (1 instance). From Flyinointment (Soud et al., 2024), we include the concrete Greedy Contract example categorized as a logic weakness, as it falls within our numeric logic error scope.

Web3Bugs (81 instances). Web3Bugs (Zhang et al., 2023) systematically collected and classified vulnerabilities reported in Code4rena audit contests. We select cases that fall within IntentChecker’s multi-contract, single-transaction analysis scope from (i)

Table 5: Benchmark dataset composition.

Source	Error Pattern	Count
NumScout	Div In Path	1
	Exchange Problem	2
	Operator Order Issue	1
	Precision Loss Trend	2
	Minor Amount Retention	1
Flyinointment	Greedy Contract	1
Web3Bugs	Erroneous Accounting	72
	Inconsistent State Updates	9
Total (collected)		89
Excluded (see below)		-15
Effective		74

categories S6-1 through S6-4 (Erroneous Accounting) and (ii) category S3-1 (Inconsistent State Updates). For S3-1, we retain only cases whose root cause involves a numeric computation with a missing or stale state-variable update. The resulting Web3Bugs subset contains 81 instances: 72 Erroneous Accounting cases and 9 Inconsistent State Updates cases.

Exclusions. Of the 89 collected instances, 15 are excluded after manual inspection because they do not satisfy our numeric-logic-error definition, fall outside the study scope, or cannot serve as distinct and reproducible benchmark instances. The exclusion categories are:

- **E1 (Source already patched, 3).** The provided source code corresponds to a post-fix version and therefore does not contain the reported bug. During provenance verification, we also inspected underlying issue-tracker records when the compiled audit report was insufficient to establish the status of the benchmark source.
- **E2 (Integer overflow/underflow, 2).** Under Solidity ≥ 0.8 checked arithmetic, the reported operation causes the transaction to revert rather than producing an incorrect numeric value; we therefore exclude these cases from the numeric logic errors considered in this study.
- **E3 (Duplicate, 2).** Multiple audit findings refer to the same buggy code location.
- **E4 (Not a bug, 1).** The reported behavior was determined to be intentional rather than a valid defect based on the audit discussion and final adjudication.
- **E5 (Missing dependency, 2).** Required external libraries are unavailable in the audit package, preventing reconstruction of the code required for analysis.
- **E6 (Multi-transaction, 3).** The reported bug requires state changes across separate transactions and therefore falls outside IntentChecker’s single-transaction analysis scope.
- **E7 (Language semantics, 1).** The reported behavior follows directly from Solidity’s specified integer-division semantics, without an erroneous numeric implementation that violates a distinct intended relation; we therefore do not treat it as a numeric logic error under Definition 1.

- **E8 (Misidentified target, 1).** Provenance inspection showed that the reported defect resides in a different contract than the benchmark target, so the provided target does not contain the reported bug and cannot serve as a reproducible instance.

The remaining **74 instances** constitute the effective evaluation set.

4.2 RQ1: Expressibility and Validatability

RQ1 examines two distinct capabilities of IntentChecker. First, we ask whether the bug-relevant numeric intent associated with a reported numeric logic error can be represented in IntentChecker’s current intent model. Second, for the cases whose intent is expressible, we evaluate whether the analyzer can actually validate the specified relation against the buggy implementation. Separating these two questions is important because a relation may be representable in the annotation language even when abstract interpretation lacks sufficient precision to establish its violation.

The evaluation is retrospective. For each benchmark instance, we reconstruct the bug-relevant intended numeric behavior using the audit report, the buggy source code, and, where available, the recommended or implemented fix. These sources serve only to establish the behavior to be specified; the fix is not mechanically translated into an annotation. Instead, we derive the least implementation-specific numeric relation that is supported by the reported intended behavior and is sufficient to distinguish the buggy behavior from the intended one. Accordingly, RQ1 evaluates the representational and validation capabilities of IntentChecker given an appropriate numeric intent specification; it does not establish that developers who are unaware of the reported bug would independently formulate the same specification.

4.2.1 Expressibility of Numeric Intent

We first determine whether the reconstructed numeric intent can be represented using IntentChecker’s current annotation model. A finding is considered expressible when all independently required bug-relevant numeric obligations can be represented using values available at supported observation points. The selected relation may take the form of an equality, inequality or bound, an Entry/Exit or Before/After relation, a state-change property, a return-value relation, or a relation over call arguments. We use **@During** when the intended property concerns an intermediate program state or statement-level effect, and **@Post** when it concerns the function’s entry/exit relationship or final state.

Expressibility is assessed independently of the subsequent abstract-interpretation result. In particular, a relation remains expressible even if the analyzer later returns *Warning* because the abstract state is too imprecise to establish whether the relation is satisfied or violated.

Of the 74 eligible benchmark instances, the bug-relevant numeric intent is expressible for **46 (62.2%)** and inexpressible for **28 (37.8%)**. Table 6 provides the breakdown by defect type. We classify a defect as *Value-level* when the intended computation is structurally present but contains an incorrect operand, operator, scaling term, or scalar assignment. A defect is *Algorithm-level* when the error involves a missing or additional operation, a missing state update, or an incorrect ordering of operations.

Table 6: Expressibility of bug-relevant numeric intent by defect type (74 eligible instances).

Error type	Expressible	Inexpressible	Total
Value	29	12	41
Algorithm	17	16	33
Total	46	28	74

4.2.2 Characterizing the Expressibility Boundary

The results in Table 6 indicate that expressibility is not limited to errors in individual arithmetic expressions. Value-level defects are expressible in 29 of 41 cases (70.7%), while 17 of 33 Algorithm-level defects (51.5%) are also expressible. Thus, missing or misordered operations can still be captured when their intended effect can be reduced to an observable numeric relation, such as a required state transition or an ordering-sensitive property over values available at the relevant program point.

To identify what prevents representation in the remaining cases, we classify the 28 inexpressible findings according to four blockers. The categories describe the specific representational obstacle rather than the developer knowledge required to discover the bug or the precision of the subsequent analysis.

- α : a required value is obtainable only through a function call inside `intentValue`, and neither a known-bound substitution nor exact formula inlining can recover it;
- β : no in-scope reference to a required bug-relevant value exists;
- γ : the intended property is inherently structural or multi-point and cannot be reduced to the supported numeric relation forms; and
- δ : the only viable observation point is architecturally unavailable to intent evaluation, such as a `@During` annotation whose required attachment point lies inside a loop body that the current engine never evaluates for intents.

Table 7 summarizes the observed blockers. A finding may contain more than one independently applicable blocker; therefore, the counts are not mutually exclusive and sum to more than 28.

These results suggest that the main boundary of IntentChecker’s current intent model is not simply whether the underlying defect is Value- or Algorithm-level. Rather, expressibility depends on whether the reported intended behavior can be reduced to numeric relations over program-observable values at supported observation points. Algorithm-level defects remain representable when their missing or misordered behavior has an observable numeric consequence, whereas even comparatively simple Value-level defects may become inexpressible when a load-bearing value is unavailable to the annotation model.

We additionally classify every eligible finding, expressible or not, as *Usable* when every value the intended relation depends on is directly referenceable at a supported program point, and *Unusable* otherwise—a purely representational question about

Table 7: Reasons for inexpressibility among the 28 inexpressible findings. Blocker counts may overlap.

Blocker	Description	Findings
α	Required value is available only through a function call in <code>intentValue</code> , with no applicable rescue	17
β	No in-scope reference to a required value exists	9
γ	Required relation is inherently structural or multi-point	5
δ	Required observation point is architecturally unavailable	2

Table 8: Value/Algorithm defect type against Usable/Unusable, across all 74 eligible findings.

Error type	Usable	Unusable	Total
Value	31	10	41
Algorithm	19	14	33
Total	50	24	74

value availability, independent of the relation’s form. Of the 74 eligible findings, **50 are Usable and 24 are Unusable** (Table 8).

Usable and Expressible coincide almost exactly, but not quite: all 46 expressible findings are Usable, and all 24 Unusable findings are inexpressible, yet 4 of the 28 inexpressible findings are still Usable—every required value is referenceable, but the relation cannot be represented for another reason. These 4 findings are tagged exclusively γ or δ , never α or β . This is consistent with how the blockers are defined: α and β are, by construction, failures of value availability and therefore always coincide with Unusable, whereas γ and δ concern the supported relation forms and observation points rather than which values are in scope, and so can leave a finding Usable even when it remains inexpressible.

4.2.3 Validation of Expressible Intent

We next evaluate whether the 46 expressible specifications can be validated by the actual analyzer. For each finding, we insert the constructed target annotation into the buggy implementation and provide batch annotations that instantiate a bug-discriminating abstract state. We then execute `IntentChecker` and record the result of the target intent check. These outcomes are obtained from actual executions of the analyzer rather than inferred from its implementation.

Table 9: Validation outcomes for the 46 expressible findings.

Validation outcome	Findings	% of expressible
Violated	32	69.6%
Warning	13	28.3%
Unsupported	1	2.2%
Total	46	100.0%

Table 10: Validation outcomes by defect type among the 46 expressible findings.

Error type	Violated	Warning	Unsupported	Total	% Violated
Value	18	10	1	29	62.1%
Algorithm	14	3	0	17	82.4%
Total	32	13	1	46	69.6%

We classify the validation results into three outcomes. *Violated* indicates that IntentChecker establishes that the buggy execution violates the specified numeric intent. *Warning* indicates that the intent is expressible but the abstract state is insufficiently precise to establish either satisfaction or violation. *Unsupported* is used when the relation itself is expressible and the observation point is valid, but the current execution model cannot represent a state transition required to perform a discriminating validation.

As shown in Table 9, **32 of the 46 expressible findings (69.6%)** produce a definite *Violated* result. A further **13 (28.3%)** produce *Warning*, while **1 (2.2%)** is *Unsupported*.

The single Unsupported finding depends on two independently required numeric mechanisms. One mechanism can be validated and produces *Violated*, whereas the other depends on an implicit change to the contract’s ETH balance caused by an external call. IntentChecker’s execution model does not model this external-call-induced balance update and therefore cannot construct the state transition required to distinguish the buggy computation from the intended relation. Under our finding-level completeness rule, we consequently classify the finding as *Unsupported* overall rather than counting the successfully validated mechanism alone as a finding-level *Violated* result.

Table 10 further breaks down the validation outcomes by defect type. Algorithm-level findings produce a definite violation in 14 of 17 cases (82.4%), whereas Value-level findings do so in 18 of 29 cases (62.1%). Both groups therefore contain a majority of definite violations, although the observed difference is descriptive and should not be interpreted as evidence of a general statistical difference between the two defect classes.

Overall, RQ1 shows that IntentChecker can represent the bug-relevant numeric intent for 46 of the 74 eligible findings. Among these expressible findings, 32 yield a definite violation when analyzed against the buggy implementation. The remaining cases expose a separate boundary of the approach: even when an intended relation can be stated in the intent language, abstract-interpretation precision or unmodeled execution semantics may prevent a definite validation result. These results therefore distinguish the representational capacity of the intent model from the validation capability of the analyzer itself.

4.3 RQ2: Specification Requirements and Analysis Costs

RQ2 characterizes the requirements of specifying numeric intent and the computational cost of validating it. We intentionally separate these two aspects. The first concerns the amount and scope of program or domain information required to formulate a bug-relevant intent specification. The second concerns the runtime cost of validating that specification once the required analysis inputs have been supplied.

We do not interpret the structural metrics reported below as direct measurements of developer effort. Instead, they characterize the specification profile of each finding: how many program elements are load-bearing to the selected relation, how far the required context extends beyond the target statement, and whether additional protocol-specific semantics are needed. These dimensions are reported separately rather than collapsed into a single weighted effort score.

4.3.1 Specification Requirements

For each of the 46 expressible findings, we identify the program context that is load-bearing to the derivation or justification of the selected numeric intent. Following the dependency rules described in Section 3, we count: (i) relevant statements, (ii) unique relevant program values, (iii) additional functions whose semantics must be inspected, and (iv) additional protocol- or application-specific contracts or libraries whose behavior is required to justify the relation.

A called function is counted once as an additional function rather than recursively expanding its internal statements into the target function’s statement count. Likewise, generic language or library semantics are recorded as supporting context when necessary but do not increase the protocol-specific contract/library count. Because a small number of findings require substantially broader context than the rest, we report medians and interquartile ranges (IQRs) rather than means.

Table 11 summarizes the resulting specification profile. The median expressible finding depends on four relevant statements and six unique program values. Most findings require no additional function beyond the target function itself, and the median number of additional protocol-specific contracts or libraries is zero.

To capture not only how much context is required but also how far that context extends from the target statement, we use the five-level context-breadth scale defined in Section 3. Level 0 requires only the target statement or expression; Level 1 requires additional context within the same function; Level 2 requires another function in the

Table 11: Specification profile across the 46 expressible findings.

Metric	Median	IQR	Min–Max
Relevant statements	4	2–6	1–20
Unique relevant values	6	4–9	1–16
Additional functions required	0	0–1	0–3
Additional protocol-specific contracts/libraries	0	0–0	0–2

Table 12: Context breadth across the 46 expressible findings.

Breadth	Meaning	Findings	%
0	Target statement/expression only	2	4.3%
1	Same-function context	21	45.7%
2	Other function in same contract/library	16	34.8%
3	Cross-contract/library context	6	13.0%
4	External protocol/domain specification	1	2.2%
Local (0–1)		23	50.0%
Intra-contract (2)		16	34.8%
Cross-boundary (3–4)		7	15.2%

same contract or library; Level 3 requires cross-contract or cross-library context; and Level 4 requires external protocol- or domain-specific semantics.

As shown in Table 12, 23 of 46 findings (50.0%) can be specified using only target-statement or same-function context (Levels 0–1). A further 16 findings (34.8%) require another function within the same contract or library (Level 2). Only 7 findings (15.2%) require cross-boundary context, i.e., another contract/library or external protocol/domain semantics (Levels 3–4).

The context-breadth score and the external-specification indicator capture related but distinct properties. Context breadth records the widest category of context required by the specification, whereas the external-specification indicator records whether any load-bearing protocol- or domain-specific semantic fact is required to justify the selected relation. Consequently, a finding may depend on external protocol semantics without necessarily receiving the maximum breadth score if its program context is otherwise narrower.

External protocol- or domain-specific semantics are required in only **5 of 46 findings (10.9%)**. The remaining **41 findings (89.1%)** can be justified from the program source together with ordinary Solidity and generic library semantics. The audit report itself is not counted merely because it is consulted during retrospective intent reconstruction; however, a protocol-specific semantic fact that is load-bearing to the selected relation and unavailable from the local source is counted as external specification.

Table 13: External-specification dependence across the 46 expressible findings.

External specification required	Findings	%
No	41	89.1%
Yes	5	10.9%

Table 14: Annotation multiplicity across the 46 expressible findings.

Annotation form	Findings	%
Single annotation	37	80.4%
Multi-annotation set	9	19.6%

We also record how many intent clauses are required at the finding level. A single annotation suffices for **37 of 46 findings (80.4%)**, while **9 findings (19.6%)** require a multi-annotation set. Multiple annotations are used only when the reported behavior contains independently checkable numeric obligations that cannot be captured completely by a single relation, including cases where the same bug-relevant mechanism applies independently to multiple program values.

Taken together, these results show that numeric-intent specification is usually supported by relatively local program context. Half of the expressible findings require no context beyond the target function, and fewer than one in six require cross-contract, cross-library, or external-domain context. At the same time, the upper ranges in Table 11 show that a smaller subset requires substantially broader program dependencies, indicating that specification requirements vary considerably across findings even when the final intent annotation itself is syntactically compact.

4.3.2 Validation Cost

We next measure the runtime cost of validating the 46 expressible findings after their target intents and analysis inputs have been supplied. Each finding is executed 10 times using a single thread. Dependency pre-analysis is excluded from the measured interval, and both definite-*Violated* and *Warning* executions are included. We use the per-case median as the representative latency because runtime measurements may be affected by transient system load and occasional outlier runs.

Across the 46 per-case medians, the overall median validation latency is **2.11 s**, with an IQR of **0.99–4.30 s**. The minimum per-case median is **0.39 s**, while the maximum is **12.99 s**, as summarized in Table 15.

The full per-finding distributions are provided in the supplementary material. To assess whether 10 repetitions provide a stable representative latency, we also compare cumulative medians obtained after 1, 3, 6, and 10 runs. This analysis is used only

Table 15: IntentChecker validation latency across the 46 expressible findings (10 runs per finding; single thread).

Metric	Value
Median latency across findings	2.11 s
IQR	0.99–4.30 s
Min	0.39 s
Max	12.99 s

as a stability check for the repetition protocol and is not treated as an additional performance metric.

These measurements characterize the computational cost of validating an already specified numeric intent; they do not measure the time or cognitive effort required for a developer to formulate the specification itself. RQ2 therefore distinguishes between structural specification requirements and analyzer runtime rather than combining them into a single notion of “annotation effort.” Overall, the results indicate that most expressible findings require relatively local specification context, while the subsequent validation typically completes within a few seconds under the evaluated environment.

4.4 RQ3: Comparison with Existing Detection Tools

4.4.1 Comparison of Analysis Models

4.4.2 Analysis Time

4.5 Threats to Validity

Internal validity. The annotations and case classifications in RQ1 are constructed retrospectively based on reported bugs, which introduces a risk of post-hoc bias. To reduce the influence of such prior knowledge on our analysis, we separate the construction of the target specification from the assessment of its evidence provenance. We first use each audit report to identify the erroneous behavior and the bug-relevant intended numeric relation. When the report provides a recommended fix, we use it only to clarify or corroborate the reported intended behavior, rather than treating the patch itself as the target specification. Cases for which the required relation is not expressible are excluded from the subsequent provenance analysis. For each expressible case, we fix the target annotation before assessing where sufficient evidence for deriving it is available. We examine the buggy function first, including its comments, parameters, data and control flow, and sibling branches; if this local evidence is insufficient, we then examine related functions or contracts, followed by external protocol or domain knowledge when necessary. Audit reports and reported fixes are not treated as evidence in this provenance assessment. Nevertheless, because the analysis is retrospective, the procedure cannot fully eliminate post-hoc bias or establish whether independent annotators without prior knowledge of the reported bugs would derive comparable annotations. Moreover, although each case’s classification (Expressibility,

Value/Algorithm-level, and the RQ2-A specification profile) was produced through an internal draft-and-review process, the analysis was conducted entirely by the authors without independent external verification; individual case-level judgments may therefore contain errors that a broader, independently-replicated annotation effort could surface and correct.

External validity. The final benchmark comprises 74 numeric logic error cases collected from three complementary sources: Web3Bugs, NumScout, and the Greedy Contract example presented in Fly-in-the-Ointment. These sources cover diverse types of numeric errors that arise in real-world Solidity contracts, but they do not exhaustively cover the full space of numeric logic errors. In addition, the benchmark is constrained by IntentChecker’s analysis scope: the NumScout cases are affected by the range of Solidity versions supported by IntentChecker’s front-end, and the Web3Bugs cases were selected from categories relevant to the multi-contract, single-transaction analysis addressed in this study, so bugs involving state changes across multiple transactions were excluded. As a result, the distributional characteristics observed with respect to expressiveness, evidence provenance, and engine limitations may not generalize to unsupported Solidity versions, multi-transaction behavior, or numeric error types not represented in the benchmark.

Construct validity. RQ1 evaluates two distinct properties of intent-based validation: whether a bug-relevant numeric relation can be represented by the current intent model, and whether IntentChecker can validate a supplied annotation against the buggy implementation. These properties characterize the representational and analysis capabilities of the proposed approach, but they do not directly measure its effectiveness in actual developer use. In particular, the annotations in this study are constructed retrospectively with knowledge of the reported defects. Therefore, an Expressible result shows that the bug-relevant numeric relation can be represented once that relation has been identified, and a Violated result shows that IntentChecker can expose an inconsistency once an appropriate intent specification is supplied. Neither result establishes that developers who are unaware of the reported bug would independently formulate the same or a comparable annotation, identify the relevant supporting evidence, or successfully diagnose and repair the defect. RQ2 partially addresses the specification-side dependency by characterizing the structural requirements associated with constructing and instantiating the selected intents. We report the program context required to support the specification, such as relevant statements, program values, additional functions or contracts, and context breadth, together with the analysis inputs required to instantiate the validation context, including batch annotations. These measures characterize the structural specification requirements of IntentChecker; they are not direct measurements of human annotation effort, learning curve, cognitive burden, or inter-annotator consistency. Consequently, the present evaluation cannot determine how easily developers can formulate numeric intent prospectively or how consistently independent annotators would produce comparable specifications. Evaluating these human-centered aspects would require an independent annotation or developer study.

5 Discussion

5.1 Intent Annotation Guidelines

Within the value-level intent model, the annotation grammar (§3) defines what can be written. The question that remains is when a developer should write which annotation. The 20 mitigated cases and 14 L5 expressible cases reveal a recurring pattern: the most impactful annotations express what the code does not already make explicit—an omitted state update, an unchecked relation between output and input, or the mathematical intent behind a lossy arithmetic path. Three non-obvious guidelines are distilled from this observation.

Guideline 1: Enumerate state variables a function should affect, not just those it actually affects. Developers naturally reason about the code they wrote—the variables they explicitly assigned, the calls they explicitly made. Bugs of omission arise precisely from this cognitive bias: a state variable that should have been updated was simply never mentioned. After writing a function, the developer should list the state variables whose value the function is expected to change and add at least a directional annotation (`@Post var(Entry) < var(Exit)` or `@Post changed(var, true)`) for each. Crucially, this practice depends only on the developer enumerating expected effects, not on any prior knowledge of the specific bug.

Guideline 2: Anchor output expectations to known quantities. Many functions with numeric outputs have a known relationship—a bound, an equality, or a monotonic relation—with quantities already in scope: input parameters, related state variables, or values derivable from specific input instantiations. Encoding this relationship as a `@During` or `@Post` annotation catches wrong-implementation bugs where the code runs without error but produces a directionally wrong or out-of-bound result. Examples include `@Post _balances[to] <= amount` bounding the output against the transferred amount (`RebaseProxy.mint`), `@During _principalWithdrawable <= _totalLiquidityWithdrawable` comparing two related state variables (`LenderPool.calcPrincipalWithdrawable`), and `@Post returnExpression == _balance - mapToken_tokenAmount[_token]` asserting the return against a state-derived formula (`Pools.getAddedAmount`). Such an annotation surfaces bugs through a violated relation even when the developer cannot pre-specify the exact output value.

Guideline 3: For multi-step arithmetic, annotate the result against the mathematically ideal expression, not the code as written. Solidity’s integer division truncates at every intermediate step, and the resulting precision loss depends on the order of operations. Code that reads `amount.div(12).mul(5)` may look correct, but the developer’s actual intent is the mathematical value $\lfloor \text{amount} \times 5 / 12 \rfloor$. Writing the intent annotation with the multiplication-before-division ordering—`@During result ≥ amount * 5 / 12`—causes `IntentChecker` to compare the code’s early-truncation path against a form that defers truncation to the final division, surfacing the precision gap automatically.

5.2 Annotation Authoring Burden and Future Automation

IntentChecker’s mitigation results are conditional on the developer supplying intent and debug annotations. This is by design: the entire approach is predicated on the developer making their intent explicit. Accordingly, IntentChecker requires direct authoring effort from the developer.

Effort 1: writing intent annotations. The intent model requires the developer to articulate, in the grammar of @During/@Post annotations, what the function is supposed to do. This translation from a mental model of intent into grammar-expressible assertions is itself effort: the developer must decide which properties to assert at which program points and must express each assertion within the grammar’s syntactic form.

Effort 2: supplying debug annotations for realistic execution paths. Even for code whose intent is clear, IntentChecker can only analyze the path that the developer’s debug annotations actually exercise. Debug annotations require the developer to choose representative ranges for each entry-point value the function reads. Choosing ranges that miss the buggy region produces a false negative, while choosing ranges that are too wide invites widening and produces uninformative Warning judgments. Either way, Satisfied judgments hold only within the declared ranges. This is the standard cost of any debugging paradigm—traditional debuggers also only execute the inputs the developer provides—and developers mitigate it by iteratively refining ranges when Warning results draw attention to specific sub-intervals.

Future direction: LLM-assisted annotation suggestion and auto-repair. A natural way to lower both annotation-authoring efforts is to combine IntentChecker with a large language model that observes the same source the developer is editing. For intent annotations, the LLM could generate candidate @During/@Post annotations from the partial code the developer is currently writing, conditioned on two paper-internal artifacts—the intent annotation guidelines (§5.1) as structural priors and the per-case annotated benchmark as in-grammar exemplars—and present them as suggestions the developer accepts or edits. For debug annotations, the LLM could propose representative ranges from invocation patterns visible in the project under edit and from per-pattern range exemplars in the annotated benchmark. Such annotation suggestions can reduce the manual authoring effort for both the 20 mitigated cases and the 14 L5 cases identified in RQ2 (Fig. ??). A second direction proposes in-editor code repair: when IntentChecker returns a Violated judgment, the LLM suggests edits that bring the program’s actual behavior within the developer’s declared intent, for them to evaluate before committing. Both directions are left as future work, building on this paper’s intent model, validation engine, guidelines (§5.1), and per-case annotated benchmark.

5.3 Current Limitations

Beyond the annotation authoring burden discussed in §5.2, two tool-side technical boundaries and one methodological limit of the present evaluation remain.

Contract-level intent. IntentChecker’s intent model operates on individual functions within a single transaction. A class of numeric properties, however, constitutes

contract-level intent—properties that relate quantities established by different transactions across the contract’s lifetime rather than within a single function execution. The canonical example is the ERC20 conservation property `totalSupply == sum(balances)`: any single function execution may temporarily increment one side and not the other, but across the entire transaction history the two must agree. Another example is the price oracle manipulation pattern, where the buggy behavior only manifests when one transaction sets up state that a later transaction reads. Capturing contract-level intent would require tracking state across transactions and aggregating across all balance entries, both of which lie outside the current function-level, single-transaction interval-domain analysis.

Unsupported Solidity constructs. IntentChecker’s Yul support targets arithmetic computation—add, sub, mul, div/sdiv, mod/smod, exp, comparison and bitwise operators, shifts, and the modular forms `mulmod/addmod`—which covers the cases where inline assembly is used as a faster substitute for high-precision integer arithmetic, by far the most common use case in DeFi libraries. However, hand-written Yul that uses memory and storage opcodes (`mload/mstore`, `sload/ssstore`), control flow (`if`, `for`, `switch`), or function definitions falls outside the supported subset and is conservatively treated as `⊤`. The same conservative treatment applies to a small number of Solidity constructs that the abstract interpreter does not model: `abi.decode` from raw bytes, `keccak256`, and other built-ins whose results depend on opaque cryptographic operations. Bugs whose buggy/correct distinction depends on the precise value produced by such constructs are correspondingly out of IntentChecker’s reach (these account for the L3 cases in Fig. ??).

Evaluation methodology: developer study. This paper evaluates IntentChecker through structural analysis—what the tool can and cannot detect (RQ1), what the intent model can and cannot express (RQ2), and how the tool compares with existing detectors (RQ3)—rather than through a developer study measuring perceived usability or annotation intuitiveness. A controlled study with Solidity-experienced participants would answer a complementary set of questions: whether developers find the annotation syntax natural, how long annotation authoring takes in practice, and whether the tool’s feedback leads to faster bug discovery than existing debugging workflows. Meaningful results for such a study require realistic development-time conditions (Chaliasos et al., 2024), which in IntentChecker’s current form are dominated by two confounds: the manual annotation-authoring burden described in §5.2, which is specifically what the envisioned LLM-assisted suggestion layer is designed to reduce, and the analyzer coverage gaps above, which bias task selection toward cases the tool already handles. The usability study is therefore deferred to after both the LLM-assisted annotation layer and the analyzer extensions are in place, where the study can assess the intended workflow rather than its current manual-authoring approximation.

6 Related Works

6.1 Numeric Logic Error Detection in Solidity

Logic error detection focuses on identifying discrepancies between developer intentions and actual program behavior in smart contracts.

(Eshghie et al., 2024) addresses business logic violations in smart contracts by using Dynamic Condition Response (DCR) graphs to model expected behaviors including temporal constraints, role-based access control, and inter-function dependencies. The system monitors runtime transactions against user-provided DCR models and their function mappings, generating violation reports when execution activities deviate from the specified business logic patterns.

(Stephens et al., 2021) verifies temporal properties of smart contracts by accepting user-specified Linear Temporal Logic (LTL) formulas and attack models to check both safety and liveness properties. The system not only detects violations but also generates concrete attack scenarios that demonstrate how the temporal properties can be violated, providing developers with actionable insights into potential logic errors.

(Wei et al., 2024) enables developers to specify function and address behaviors using the ConSol syntax alongside their Solidity code, then verifies whether the implementation adheres to these behavioral specifications and generates behavior-compliant Solidity programs without annotations.

(Wang et al., 2024b) leverages multimodal learning to automatically infer invariants and detect machine un-auditable bugs (MUBs) (Zhang et al., 2023) by analyzing both source code and natural language artifacts including comments, function signatures, and documented transaction scenarios. The system employs a fine-tuned model with Tier of Thought (ToT) prompting that progressively predicts transaction contexts, generates and ranks invariants, and ultimately identifies vulnerabilities through three-tier reasoning.

Furthermore, there are also approaches that address multiple transaction logic errors such as price manipulation (Wu et al., 2023; Wang et al., 2024a; Xi et al., 2024; Arora et al., 2024; Kong et al., 2023) and flash loan attacks (Li et al., 2025; Ramezany et al., 2023).

The numeric logic error detectors above and IntentChecker differ along both axes identified in §2.2: who authors the detection criteria and when the tool is run. Detectors that rely on pattern catalogs, language-model inference, or type rules are all designed to run over completed code, with criteria fixed at design time and applied post-hoc. IntentChecker is positioned earlier in time and lets the developer declare intent explicitly through annotations evaluated inside the editor, crossing both axes simultaneously. Because the two approaches see different inputs (code only vs. code with declared intent) and run in different workflow stages (post-hoc audit vs. development-time validation), their coverage is necessarily complementary.

6.2 Specification-based Smart Contract Verification

Specification-based verification approaches focus on verifying general properties and correctness requirements of smart contracts rather than specific developer intentions.

([Consensys Diligence, 2026c](#)) provides a framework that transforms developer annotations into Solidity assertion code, enabling testing with existing tools like MythX ([Consensys Diligence, 2026b](#)) and Mythril ([Consensys Diligence, 2026a](#)). The framework supports multiple annotation types including `if_succeed` (conditions for successful function returns), `invariant` (contract-level properties that must always hold), `if_updated` (conditions checked when state variables change), and `assert` (validations at specific execution points), making formal specifications accessible through familiar testing pipelines.

([Wesley et al., 2022](#)) builds upon Scribble ([Consensys Diligence, 2026c](#)) to verify whether Solidity code satisfies specifications written in the Scribble language, leveraging C-based verification tools like SeaHorn ([Gurfinkel et al., 2015](#)), libFuzzer ([LLVM, 2026](#)), and KLEE ([Cadar et al., 2008](#)) by transforming smart contracts into LLVM-IR based harnesses. The framework abstracts the 2^{160} possible user addresses into a small set of representative users (implicit, transient, and explicit), enabling verification of properties that must hold for arbitrary numbers of users, such as accounting properties like "the contract balance equals the sum of all user deposits."

([Nguyen et al., 2023](#)) verifies Solidity code using two types of annotations: specification annotations (loop invariants, call invariants, and call modifies) that developers declare as assumptions, and verification annotations (`ensures`, `reverts_if`, and contract invariants) that the tool checks against the implementation. The system assumes the truth of specification annotations and uses them to verify whether the code satisfies function post-conditions, correctly handles revert conditions, and maintains contract invariants throughout execution.

([Bartoletti et al., 2024](#)) focuses on verifying liquidity properties of smart contracts through formal methods by converting both the contract code and user-defined liquidity properties into SMT constraints. The system then checks whether the contract satisfies these properties using SMT solvers, generating counterexamples when violations are detected to help developers understand potential liquidity issues.

While the specification-based approaches above provide dedicated specification languages and discharge verification obligations through SMT solving, symbolic execution, or assertion injection over completed code, IntentChecker’s intent model occupies a different point in the design space. First, its annotation language targets value-level numeric intent expressed as interval-valued relationships over specific variables and program points, whereas specification-based languages above express properties as Boolean assertions over types or contract state. Second, its tri-valued validation engine (§3.3.1) operates over the intervals produced by interval-domain abstract interpretation, which is light enough to run at development time as the developer writes code, whereas SMT-based and symbolic-execution-based checks are discharged as verification tasks after coding is complete. Third, each annotation yields a graded outcome (Satisfied, Warning, or Violated) along with a risk score and the sub-intervals where the violation may occur, pointing the developer to the problematic value region rather than returning a binary verification verdict. The two approaches are therefore complementary in specification style, verification backend, and workflow stage.

6.3 Tools for Strengthening Security During Development Phase

Development-phase security tools integrate vulnerability detection directly into the coding workflow, enabling developers to identify and address security issues as they write code rather than in post-development analysis. (Ren et al., 2021) presents an integrated development environment for smart contract development that combines natural language model-based code autocompletion with security validation. The platform integrates existing static analysis tools including Oyente (Luu et al., 2016), Mythril (Consensys Diligence, 2026a), and Securify (Tsankov et al., 2018) to provide comprehensive security reports during development workflow.

(Protofire, 2026) provides a Solidity linter that detects predefined vulnerability patterns including low-level calls, reentrancy, and deprecated function usage during development. The tool enforces security best practices by checking for patterns such as proper visibility declarations, send result validation, and secure compiler version usage.

While these development-phase tools rely on design-time detection criteria for known vulnerability patterns, IntentChecker enables developers to specify their intended numeric behaviors through annotations and validates whether the actual program execution matches the declared intent.

7 Conclusion

This paper presents IntentChecker, a development-time value-level intent validation tool for Solidity built on SolQDebug and extended to a multi-contract single-transaction scope. The tool classifies developer-supplied @During/@Post intent annotations as Satisfied, Warning, or Violated against the abstract states produced by interval-domain analysis, grounded in a formal denotational semantics, and reports a risk score and the violated region per annotation.

On 74 effective benchmark instances drawn from Web3Bugs, NumScout, and the Greedy Contract dataset of Soud et al. (2024), IntentChecker’s intent model can represent the bug-relevant numeric relation for 46 instances (RQ1); among these, actual execution against the buggy implementation yields a Violated judgment for 32, a Warning for 13, and a single Unsupported outcome tied to an unmodeled external-call state transition. The remaining 28 instances are inexpressible, predominantly (17 of 28) because the required value is reachable only through a function call inside the annotation with no available rescue (α), with the rest attributable to values with no in-scope reference (β , 9), inherently structural relations (γ , 5), or an architecturally unevaluated observation point (δ , 2). RQ2 characterizes the specification cost of the 46 expressible instances: a median of 4 relevant statements and 6 unique program values, with half requiring no more than same-function context (breadth 0–1) and 89.1% requiring no external protocol-specific knowledge; validation itself completes at a median of 2.11 s per case. RQ3 then compared IntentChecker with existing detectors and empirically confirmed that their coverage and latency envelopes are bounded by both structural axes this paper identifies, namely who authors the detection criteria and when the tool is run.

These results position IntentChecker as complementary to existing detection tools: rather than competing on detection accuracy, it fills the development-time slot that auditing-time tools structurally cannot occupy. Future work targets LLM-assisted annotation suggestion and auto-repair, along with capturing contract-level intent.

Acknowledgements

This work was supported by the Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korean government (MSIT) (RS-2021-II210177, High Assurance of Smart Contract for Secure Software Development Life Cycle), the Technology development Program (RS-2025-25455127) funded by the Ministry of SMEs and Startups (MSS, Korea), and the “Regional Innovation System & Education (RISE)” through the Seoul RISE Center, funded by the Ministry of Education (MOE) and the Seoul Metropolitan Government (2025-RISE-01-003-01).

Author Contributions

Inseong Jeon participated in conceptualization, methodology design, system implementation, data collection, experiments, and manuscript writing. Sundeuk Kim and Hyunwoo Kim assisted in experiments, data collection and analysis, and contributed to manuscript writing. Hoh Peter In provided resources, assisted in editing the manuscript, and supervised the entire project. All authors reviewed and approved the final version of the manuscript.

Data Availability

The benchmark of 89 real-world numeric logic error instances curated from Web3Bugs (Zhang et al., 2023), NumScout (Chen et al., 2025), and the Greedy Contract dataset of Soud et al. (2024), along with the IntentChecker implementation (IntentChecker, 2026a), evaluation scripts, and experimental results, are available at <https://github.com/iwwyou/IntentChecker.git>.

Declarations

Competing interests The authors declare no competing interests.

Ethical approval Not applicable since there are no human and/or animal studies included in this paper.

References

- Arora, S., et al.: SecPLF: Secure protocols for loanable funds against oracle manipulation attacks. In: Proceedings of the 19th ACM Asia Conference on Computer and Communications Security (ASIACCS), pp. 1394–1405 (2024). <https://doi.org/10.1145/3634737.3637681>
- Anthropic: Claude. <https://www.anthropic.com/> (2026). Accessed April 2026

- Bartoletti, M., et al.: Solvent: liquidity verification of smart contracts. In: International Conference on Integrated Formal Methods (iFM), pp. 256–266 (2024). https://doi.org/10.1007/978-3-031-76554-4_14
- Cadar, C., et al.: KLEE: unassisted and automatic generation of high-coverage tests for complex systems programs. In: OSDI, pp. 209–224 (2008). <https://doi.org/10.5555/1855741.1855756>
- Chaliasos, S., et al.: Smart contract and defi security tools: Do they meet the needs of practitioners?. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE), pp. 1–13 (2024). <https://doi.org/10.1145/3597503.3623302>
- Chen, J., et al.: NumScout: Unveiling Numerical Defects in Smart Contracts using LLM-Pruning Symbolic Execution. IEEE Transactions on Software Engineering (2025). <https://doi.org/10.1109/TSE.2025.3555622>
- NumScout: 95 Samples Experiment Dataset. <https://github.com/NumScout/NumScout/tree/main/Experiment/95.Samples.Run> (2026). Accessed April 2026
- NumScout: feature detector source code (`semantic_analysis.py`). https://github.com/NumScout/NumScout/blob/main/feature_detector/semantic_analysis.py (2026). Accessed April 2026
- Consensys Diligence: Mythril. <https://github.com/ConsenSysDiligence/mythril> (2026). Accessed April 2026
- Consensys Diligence: MythX. <https://mythx.io/> (2026). Accessed April 2026
- Consensys Diligence: Scribble. <https://diligence.security/scribble/> (2026). Accessed April 2026
- Eshghie, M., et al.: Highguard: Cross-chain business logic monitoring of smart contracts. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 2378–2381 (2024). <https://doi.org/10.1145/3691620.3695356>
- Feist, J., Grieco, G., Groce, A.: Slither: A static analysis framework for smart contracts. In: 2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB), pp. 8–15 (2019). <https://doi.org/10.1109/WETSEB.2019.00008>
- Gurfinkel, A., et al.: The SeaHorn verification framework. In: International Conference on Computer Aided Verification, pp. 343–361 (2015). https://doi.org/10.1007/978-3-319-21690-4_20
- Google: Gemini. <https://gemini.google.com/> (2026). Accessed April 2026
- Jeon, I., et al.: SolQDebug: Debug solidity quickly for interactive immediacy in smart contract development. Automated Software Engineering 33(3), 86 (2026). <https://doi.org/10.1007/s10515-026-00618-9>
- IntentChecker. <https://github.com/iwwyou/IntentChecker.git> (2026). Accessed April 2026
- IntentChecker: DAppSCAN contract analysis dataset. https://github.com/iwwyou/IntentChecker/blob/main/evaluation/DappSCAN_contract_analysis/contract_analysis_table.xlsx (2026). Accessed April 2026

- IntentChecker: RQ2 supplementary material—L4/L5 case classifications and annotations. <https://github.com/iwyyou/IntentChecker/tree/main/evaluation/RQ2> (2026). Accessed April 2026
- IntentChecker: RQ1 supplementary material—per-case annotation plans and rationale. https://github.com/iwyyou/IntentChecker/tree/main/evaluation/RQ1/annotation_plans.md (2026). Accessed April 2026
- Kong, Q., et al.: Defitainter: Detecting price manipulation vulnerabilities in defi protocols. In: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), pp. 1144–1156 (2023). <https://doi.org/10.1145/3597926.3598124>
- Li, X., et al.: Penetrating the hostile: Detecting defi protocol exploits through cross-contract analysis. *IEEE Transactions on Information Forensics and Security* 20, 11759–11774 (2025). <https://doi.org/10.1109/TIFS.2025.3626979>
- LLVM: libFuzzer. <https://lvm.org/docs/LibFuzzer.html> (2026). Accessed April 2026
- Luu, L., et al.: Making smart contracts smarter. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 254–269 (2016). <https://doi.org/10.1145/2976749.2978309>
- Nguyen, T.D., et al.: An idealist’s approach for smart contract correctness. In: International Conference on Formal Engineering Methods, pp. 11–28 (2023). https://doi.org/10.1007/978-981-99-7584-6_2
- OpenAI: ChatGPT. <https://chatgpt.com/> (2026). Accessed April 2026
- Pearson correlation coefficient. https://en.wikipedia.org/wiki/Pearson_correlation_coefficient (2026). Accessed April 2026
- Protofire: Solhint. <https://github.com/protofire/solhint> (2026). Accessed April 2026
- Ramezany, S., Setthawong, R., Setthawong, P.: Midnight: An Efficient Event-driven EVM Transaction Security Monitoring Approach For Flash Loan Detection. In: Proceedings of the 2023 20th International Joint Conference on Computer Science and Software Engineering (JCSSE), pp. 237–241 (2023). <https://doi.org/10.1109/JCSSE58229.2023.10201970>
- Ren, M., et al.: Making smart contract development more secure and easier. In: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), pp. 1360–1370 (2021). <https://doi.org/10.1145/3468264.3473929>
- Soud, M., Liebel, G., Hamdaqa, M.: A fly in the ointment: an empirical study on the characteristics of Ethereum smart contract code weaknesses. *Empirical Software Engineering* 29(1), 13 (2024). <https://doi.org/10.1007/s10664-023-10398-5>
- Stephens, J., et al.: SmartPulse: automated checking of temporal properties in smart contracts. In: Proceedings of the 2021 IEEE Symposium on Security and Privacy (SP), pp. 555–571 (2021). <https://doi.org/10.1109/SP40001.2021.00085>
- Sun, Y., et al.: Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE), pp. 1–13 (2024). <https://doi.org/10.1145/3597503.3639117>

- GPTScan: Web3Bugs per-contest evaluation results. <https://github.com/MetaTrustLabs/GPTScan-Web3Bugs> (2023). Accessed April 2026
- Tsankov, P., et al.: Securify: Practical security analysis of smart contracts. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 67–82 (2018). <https://doi.org/10.1145/3243734.3243780>
- Wang, D., et al.: Defiguard: A price manipulation detection service in defi using graph neural networks. *IEEE Transactions on Services Computing* (2024). <https://doi.org/10.1109/TSC.2024.3489439>
- Wang, S.J., Pei, K., Yang, J.: Smartinv: Multimodal learning for smart contract invariant inference. In: Proceedings of the 2024 IEEE Symposium on Security and Privacy (SP), pp. 2217–2235 (2024). <https://doi.org/10.1109/SP54263.2024.00126>
- Wei, G., et al.: Consolidating Smart Contracts with Behavioral Contracts. Proceedings of the ACM on Programming Languages 8(PLDI), 965–989 (2024). <https://doi.org/10.1145/3656416>
- Wesley, S., et al.: Verifying Solidity smart contracts via communication abstraction in SmartACE. In: International Conference on Verification, Model Checking, and Abstract Interpretation, pp. 425–449 (2022). https://doi.org/10.1007/978-3-030-94583-1_21
- Wu, S., et al.: Defiranger: Detecting defi price manipulation attacks. *IEEE Transactions on Dependable and Secure Computing* 21(4), 4147–4161 (2023). <https://doi.org/10.1109/TDSC.2023.3346888>
- Xi, R., Wang, Z., Pattabiraman, K.: POMABuster: Detecting Price Oracle Manipulation Attacks in Decentralized Finance. In: Proceedings of the 2024 IEEE Symposium on Security and Privacy (SP), pp. 3923–3942 (2024). <https://doi.org/10.1109/SP54263.2024.00257>
- Zhang, B.: Towards finding accounting errors in smart contracts. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE), pp. 1–13 (2024). <https://doi.org/10.1145/3597503.3639128>
- Zhang, W., et al.: Nyx: Detecting exploitable front-running vulnerabilities in smart contracts. In: Proceedings of the 2024 IEEE Symposium on Security and Privacy (SP), pp. 2198–2216 (2024). <https://doi.org/10.1109/SP54263.2024.00146>
- Yang, S., et al.: Hyperion: Unveiling DApp Inconsistencies Using LLM and Dataflow-Guided Symbolic Execution. In: Proceedings of the 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), pp. 2125–2137 (2025). <https://doi.org/10.1109/ICSE55347.2025.00015>
- Zhang, Z., et al.: Demystifying exploitable bugs in smart contracts. In: Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 615–627 (2023). <https://doi.org/10.1109/ICSE48619.2023.00061>
- Zheng, Z., et al.: Dappscan: building large-scale datasets for smart contract weaknesses in dApp projects. *IEEE Transactions on Software Engineering* 50(6), 1360–1373 (2024). <https://doi.org/10.1109/TSE.2024.3383422>
- Zhou, L., et al.: Sok: Decentralized finance (defi) attacks. In: Proceedings of the 2023 IEEE Symposium on Security and Privacy (SP), pp. 2444–2461 (2023).

<https://doi.org/10.1109/SP46215.2023.10179435>

A Validation Engine: Class and Sequence Diagrams

This appendix details the internal structure of Phase 3, the Validation Engine (§3), which is this paper’s core contribution. Figure 7 shows the classes involved and their relationships; Figure 8 traces the message flow for a single analysis run, from the point the Parsing and Analysis Modules (Phases 1–2, reused from SolQDebug (Jeon et al., 2026)) hand off a control-flow graph and computed intervals to the point a line-level **Result/Risk Score/Violated Region** is produced. To keep the diagrams readable and consistent with this paper’s own terminology, class and method names below are abstracted rather than copied verbatim from the source; Table 16 (§A.3) gives the exact correspondence to the public implementation (IntentChecker, 2026a).

A.1 Class Diagram

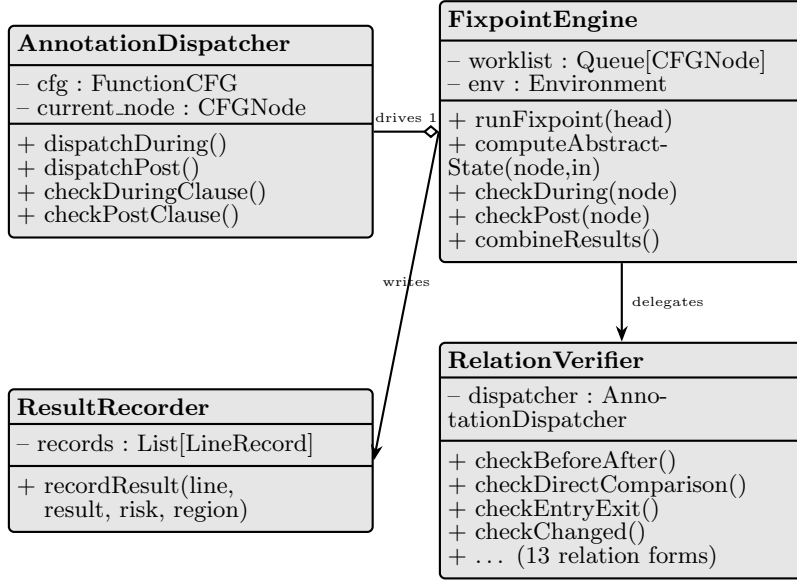


Fig. 7: Class diagram of the Validation Engine (Phase 3). **ANNOTATIONDISPATCHER** (Phases 1–2) drives one **FIXPOINTEngine** instance per function analysis; **FIXPOINTEngine** performs the interval-domain fixpoint computation and, at each eligible CFG node, delegates the actual per-relation-form check to **RELATIONVERIFIER**, then writes the combined outcome through **RESULTRECORDER**. Names here are paper-facing abstractions — see Table 16 for the exact implementation identifiers.

AnnotationDispatcher owns the function’s control-flow graph and dispatches each parsed **@During/@Post** clause to the Validation Engine. **FixpointEngine** is the interval-domain abstract interpreter: **runFixpoint** drives a worklist over the CFG,

and `computeAbstractState` computes each node's abstract state. At every CFG node the fixpoint computation actually visits, `checkDuring`/`checkPost` invoke **RelationVerifier**, which implements one method per relation form from §4.2.1 (e.g., `checkBeforeAfter` for Before-After relations, `checkEntryExit` for Entry-Exit relations, and ten further methods covering the remaining During/Post relation forms). Each call returns a three-valued outcome (Satisfied/Violated/Warning) plus, when applicable, the sub-interval responsible for a Warning. When a clause is a compound `&&/||` expression, `FixpointEngine.combineResults` combines the member outcomes; the result is then handed to **ResultRecorder**, which stores the final per-line **Result**, **Risk Score**, and **Violated Region**.

A.2 Sequence Diagram

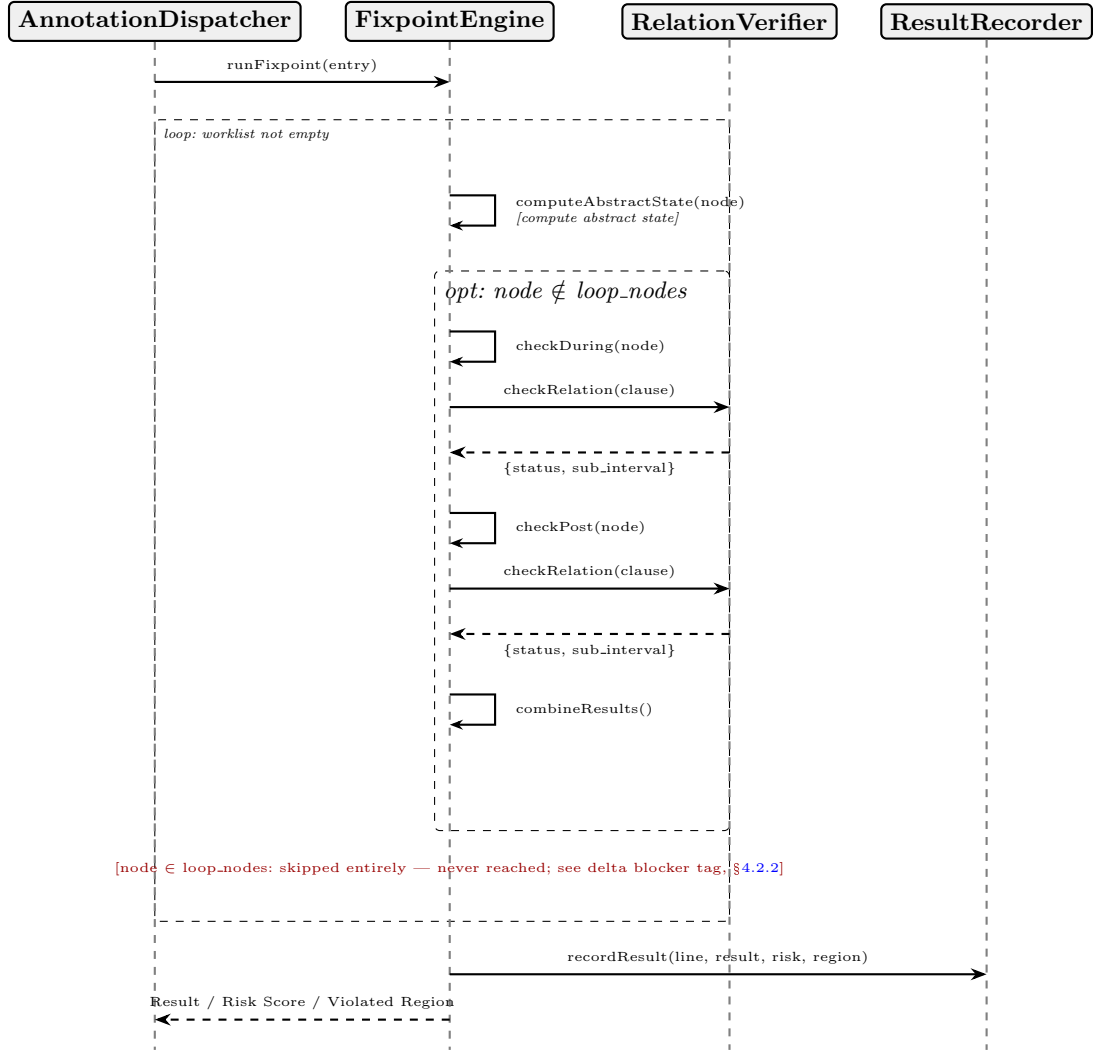


Fig. 8: Sequence diagram of one Validation Engine analysis run. The outer loop is the fixpoint worklist iteration; the inner *opt* frame fires only for CFG nodes the engine’s per-node checking machinery actually visits. Nodes inside a loop body (*loop_nodes*) never enter this frame at all — *computeAbstractState* computes their abstract state but *checkDuring*/*checkPost* are never invoked for them, which is the architectural fact underlying the *delta* blocker tag (§4.2.2).

Figure 8 makes explicit a fact that is otherwise only visible by reading the implementation directly (Table 16): the per-node intent-checking step (the inner `opt` frame) is conditioned on the node not being a loop-interior node. The fixpoint worklist still computes an abstract state for every node, loop-interior or not (`computeAbstractState` always runs), but `checkDuring/checkPost` — and therefore every downstream call into `RELATIONVERIFIER` — are simply never invoked for a loop-interior node. Consequently, an annotation whose only viable attachment point is inside a loop body is not merely analyzed imprecisely; it is never evaluated at all, independent of the annotation’s own content. This is the source of the *delta* category in §4.2.2’s expressibility-boundary taxonomy, and is otherwise easy to conflate with an ordinary precision limitation (widening, $\top$ -propagation) rather than the architectural non-invocation it actually is.

A.3 Correspondence with the Public Implementation

Table 16 maps every paper-facing name in Figures 7–8, and the Phase 1–3 terminology used in §3, to the corresponding file and identifier in the public repository ([IntentChecker, 2026a](#)). The implementation predates this paper’s `IntentChecker` naming and retains its original project name, `SOLIDITYGUARDIAN`, in some internal identifiers (e.g., the `Guardian` prefix below); this does not reflect a different or additional tool.

Table 16: Paper terminology to implementation correspondence. All paths are relative to the repository root ([IntentChecker](#), 2026a).

Paper term	Implementation
<i>Phases</i>	
Parsing Module (Phase 1)	Parser/, Analyzer/ContractParser.py
Analysis Module (Phase 2)	Interpreter/Engine.py, Analyzer/DynamicCFGBuilder.py, Analyzer/StaticCFGFactory.py
Validation Engine (Phase 3)	Analyzer/ContractAnalyzer.py, Interpreter/Engine.py, Analyzer/GuardianVerificationEngine.py, Analyzer/RecordManager.py
<i>Classes</i>	
ANNOTATIONDISPATCHER	Analyzer/ContractAnalyzer.py :: ContractAnalyzer
FIXPOINTENGINE	Interpreter/Engine.py :: Engine
RELATIONVERIFIER	Analyzer/GuardianVerificationEngine.py :: GuardianVerificationEngine
RESULTRECORDER	Analyzer/RecordManager.py :: RecordManager
<i>Methods</i>	
dispatchDuring() / dispatchPost()	ContractAnalyzer.process_during / .process_post
checkDuringClause() / checkPostClause()	ContractAnalyzer._verify_during_clause / ..verify_post_clause
runFixpoint(head)	Engine.fixpoint(head)
computeAbstractState(node, in)	Engine.transfer_function(node, in_vars)
checkDuring(node) / checkPost(node)	Engine._process_during_annotations / ..process_post_annotations
checkRelation(clause)	GuardianVerificationEngine.verify_during_* / ..verify_post_* (13 methods, e.g. verify_during_before_after, verify_post_entry_exit, verify_post_changed)
combineResults()	Engine._combine_logic_results
recordResult(...)	RecordManager.record_result